

UNIVERSITI TUNKU ABDUL RAHMAN

ACADEMIC YEAR 2025



Wholly owned by UTAR Education Foundation
(Co. No. 578227-M)
DU012(A)

UCCD 1004 PROGRAMMING CONCEPTS AND PRACTICES

ASSIGNMENT 2

Group 23			
Name	ID	Programme	UTAR Email
Ong Jun Kit	24ACB00274	CS	junkit040@lutar.my
Tai Tiong Jun	24ACB00192	CS	tiongjun73@lutar.my
Wong Hoe Mun	24ACB00164	CS	wonghoward@lutar.my

Task Division

	Name	Role	Modules
1.	Ong Jun Kit	Combine Code	readPaymentCheckIn
			payment
			checkIn
2.	Tai Tiong Jun	Fix Errors	readBooking
			performBooking
			editBooking
3.	Wong Hoe Mun	Error Checking	readUser
			registration
			login

Table of Content

Section	Content	Page
A	Objective	1
B	Pseudocode	1 - 8
C	Flowchart	9 - 35
D	Test Cases	36 - 42
E	Source Code	43 - 100

A. Objective

To develop and set up an online flight booking system for JSJK Airlines that allows users to easily manage their travel plans by selecting flights, booking tickets, making safe payments, and checking in online. The objective of the system is to improve overall efficiency and customer satisfaction by giving users access to important travel documents like invoices.

B. Pseudocode

1.0 Main Menu

Start

Do and repeat while the user wants to continue.

Display "WELCOME TO JSJK AIRLINE"

Display menu,

1. Register

2. Login

3. Quit

Prompt the user to select the options.

If user select choice1 = 1 to register,

Call registration()

Else if user select choice1 = 2 to login,

Call readUser(users, userCount)

Call login(users, userCount)

IF no error,

Display "Login Successful"

Do and repeat it while true:

Display booking system menu:

1.Book Ticket

2.Edit Booking

3.Perform Transaction

4.Check In

5.Print Invoice

6.Quit

Call readBooking function.

noTickets = readBooking(currentUser, bookings)

Call readPaymentCheckIn(UserCheckIn) function.

Prompt the user to select options.

Validate option between 1 and 6.

If user select choice2 = 1 to book ticket,

Call performBooking()

Else if user select choice2 = 2 to edit ticket,

```

        Call editBooking()
    Else if user select choice2 = 3 to perform transaction,
        If noTickets = 0, display "No bookings available".
        Else, call payment()
    Else if user select choice2 = 4 to check in,
        Call readPaymentCheckIn(UserCheckIn)
        Call checkIn(UserCheckIn)
    Else if user select choice2 = 5 to print invoice,
        If noTickets = 0, display "No bookings found".
        Else, call readPaymentCheckIn(UserCheckIn) and printInvoice()
    Else if user select choice2 = 6 to quit,
        Break from booking loop

    Prompt the user "Press Enter to return to booking menu"

    If select choice2 = 1 or 2,
        Create the current user's new empty paymentCheckIn and Invoice
        text file and close.
    Clear the screen.

    Else if error = 1,
        Display "Username NOT FOUND"
    Else if error = 2,
        Display "Incorrect Password"

Else if choice1 = 3,
    Display "Thank you. Goodbye."
    Exit program

If choice1 = 1 or 2,
    Prompt user "Do you want to continue login? (Y/N)"

```

END

1.1 readUser(Users, userCount)

START

Open file user.txt for reading.
Set userCount to 0 for initialize the number of users.

```

Loop until end of file or 100 users:
    If cannot read fullname.name1 from file into users[userCount],
        Break the looping.
    Read fullname.name2 from file into users[userCount]
    Read phone from file into users[userCount]
    Read email from file into users[userCount]
    Read username from file into users[userCount]
    Readpassword from file into users[userCount]

```

Increment userCount by 1
Close file user.txt

END

1.2 userRegistration

START

Display "Register New User".

Prompt the user to enter fullname.name1.

Prompt the user to enter fullname.name2.

Prompt the user to enter phone.

Prompt the user to enter email.

Prompt the user to enter username.

Do and repeat until password is valid:

 Prompt the user to enter password.

 If password length less than 8,

 Display "Password too short".

 Continue the loop,

 Check if password contains:

 At least one uppercase letter.

 At least one digit.

 At least one symbol.

 If any condition not met,

 Display missing requirements.

 Continue the looping,

 If all conditions met,

 Break the looping.

Display "Registration Successful".

Open file user.txt in append mode,

Write user details fullname.name1, fullname.name2, phone, email, username line-by-line into file.

Close file user.txt.

END

1.3 userLogin

START

Display "Login".

Prompt user to enter username1.

Prompt user to enter password1.

For each user from 0 to userCount – 1:

 If users[i].username = username1,

 If users[i].password = password1,

 Set username1 as currentUser .

 Return 0

 Else return 2.

Return 1.

END

1.4 readBooking

START

Open file currentUser and Booking.txt for reading.
Set total to 0.

While there is still have data in the file:

- If the attempt to read passengerName.name1 into bookings[total] fails, exits the loop.
- If the attempt to read passengerName.name2 into bookings[total] fails, exits the loop.
- If the attempt to read depart_flight into bookings[total] fails, exits the loop.
- Ignore the newline character.
- If the attempt to read date_depart into bookings[total] fails, exits the loop.
- If the attempt to read time_depart into bookings[total] fails, exits the loop.
- Ignore the newline character.
- If the attempt to read return_flight into bookings[total] fails, exits the loop.
- Ignore the newline character.
- If the attempt to read date_return into bookings[total] fails, exits the loop.
- If the attempt to read time_return into bookings[total] fails, exits the loop.
- Ignore the newline character.

- Increment total by 1.
- If total larger or equal to SIZE break.

Close file.

Return total.

END

1.5 Perform Booking

START

Display available flight schedules and time slots using FlightSchedule().
Prompt the user to enter noTickets.

IF noTickets less than or equal to 0 or noTickets larger than SIZE,
Display error and return.

Open file named currentUser + '_Booking.txt' for writing.

For I = 0 to noTickets - 1:

- Display "Passenger i+1".
- Prompt user and read passengername.name1 into b[i].
- Prompt user and read passengername.name2 into b[i].
- Prompt user and read depart_flight into b[i].
- Prompt user and read date_depart into b[i].
- Prompt user and read time_depart into b[i].
- Prompt user and read return_flight into b[i].
- Prompt user and read date_return into b[i].
- Prompt user and read time_return into b[i].

Write all details of passenger line-by-line into booking file.

Close booking file.

Display "booking completed".

END

1.6 editBooking

Start

Call readBooking(currentUser, b)

Total = readBooking(currentUser, b)

If total = 0:

 Display "No booking found" and return.

Prompt user "Change passenger name? (y/n)" to change name.

If user wants to change name,

 Ignore the newline character.

 For i = 0 to total – 1,

 Display current name for passenger i+1.

 Prompt user and read new first name.

 Prompt user and read new last name.

Do and repeat until user select exit,

 Display edit menu with options 1 to 7.

 Prompt user enter the choice.

 Validate the choice between 1 to 7.

 If choice not equal to 7,

 Display old value of selected field.

 Prompt user to enter new value.

 Update details in b[0]

 If user select choice = 7,

 Return.

For each passenger i = 0 to total – 1,

 Copy updated fields from b[0] to b[i],

Prompt user "confirm editing? (y/n)" for confirmation.

If user wants to confirm editing,

 Open file currentuser and booking.txt for writing.

 For i = 0 to total – 1,

 Write all booking info from b[i] to file.

Close file.

Display "Booking successfully updated".

Prompt user to press enter to return to menu
Clear screen.

END

1.7 readPaymentCheckIn

START

OPEN file currentUser + '_paymentCheckIn.txt' for reading.

If file open successfully,

 Read totalAmount into UserCheckIn.

 Read paymentStatus into UserCheckIn.

 For i = 0 to noTickets – 1,

 Read passengerName.name1 into UserCheckIn.checkInPassengers[i].

 Read passengerName.name2 into UserCheckIn.checkInPassengers[i].

 Read passportNum into UserCheckIn.checkInPassengers[i].

 Read contactName.name1 into UserCheckIn.checkInPassengers[i].

 Read contactName.name2 into UserCheckIn.checkInPassengers[i].

 Read contactMobileNum into UserCheckIn.checkInPassengers[i].

 Read checkInStatus into UserCheckIn.

 Close file.

END

1.8 Payment

START

Calculate total amount = (price_depart + price_return) * noTickets.

Display ticket details and amount.

Prompt the user for payment method (1 = Credit/Debit Card, 2 = Bank Transfer).

If the user select method = 1,

 Prompt user for card holder name.

 Prompt and validate card number.

 Prompt and validate card expiry date.

 Prompt and validate security code.

 Prompt user to confirm transaction (Y/N).

Else if the user select method = 2,

 Display bank list.

 Prompt user to select bank.

 Prompt user and validate username.

 Prompt for password.

 Display amount and prompt to request TAC.

If user wants to confirm,
 Prompt user and validate TAC number.

 If user wants to confirm payment,
 Open file currentUser + '_paymentCheckIn.txt' for writing.
 Write amount.
 Write "Paid".

 For i from 0 to noTickets * 6 - 1 WRITE,
 Write "NULL".

 Write "Not Check".
 Close file.
 Display "Payment Successfully Completed".

END

1.9 Checkin

START

Display "Check In".

If UserCheckIn.paymentStatus = "Paid" and UserCheckIn.checkInStatus = "Not Check",

 For i = 0 to noTickets - 1,
 Prompt the user and read passengerName.name1.
 Prompt the user and read passengerName.name2.
 Prompt the user and read passportNum and validate format.

 If i not equal to 0,
 Prompt user "Use previous contact info? (Y/N)" for choice.
 If user wants to use previous contact info,
 Prompt user for previous index and copy contactName and
 contactMobileNum.
 Display contact information.

 Else prompt the user and read new contactName and contactMobileNum
 Prompt user "Confirm Check In? (Y/N)" for confirmation.

 If user want to confirm check in,
 Open file currentUser and paymentCheckIn.txt for writing.
 Write UserCheckIn.totalAmount.
 Write UserCheckIn.paymentStatus.

 For i = 0 to noTickets - 1,
 Write all details of checkInPassengers[i].
 Increment i by 1.

Write "Checked".

Close file

DISPLAY "Check-In Successfully Completed".

```
Else if UserCheckIn.checkInStatus = "Checked",
    Display "Checked In Completed".

Else
    Display "Payment Not Made. Cannot Check In".
    Display "Please proceed to make payment before check in."

END
```

1.10 printInvoice

START

Open file named currentUser and invoice.txt for writing.

Write header line "Welcome to JSJK Airline Company".

Write departure flight info:

Route from route_option[flight.depart_flight - 1].

Date from flight.date_depart.

Slot from timeSlot[flight.time_depart - 1].

Write return flight info:

Route from route_option[flight.return_flight - 1].

Date from flight.date_return.

Slot from timeSlot[flight.time_return - 1].

For each passenger i to 0 to noTickets - 1:

 Write "Passenger i+1".

 Write full name using checkInPassengers[i].passengerName.name1 and name2.

 Write passportNum.

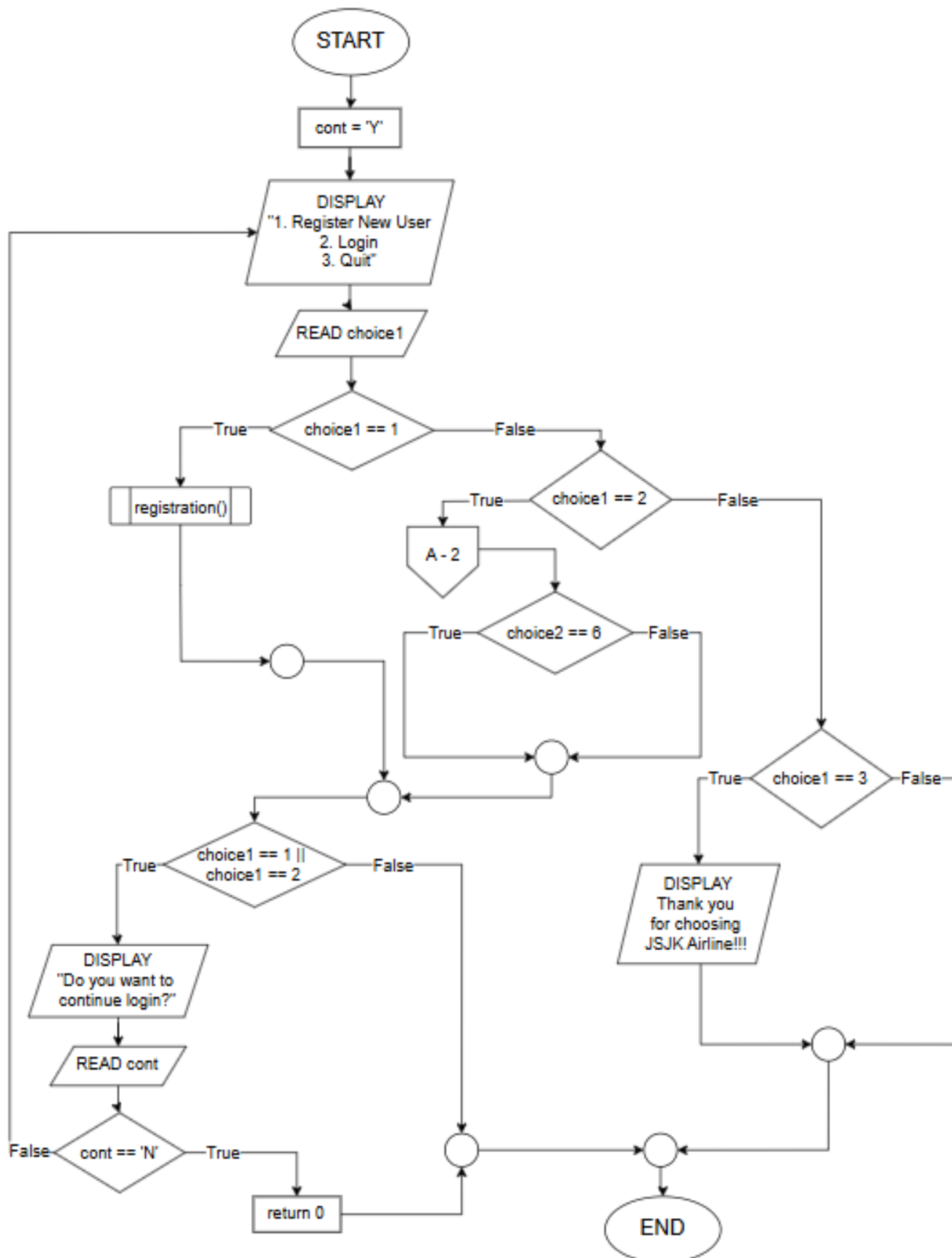
 Write contact name and mobile from contactName.name1, name2, and contactMobileNum

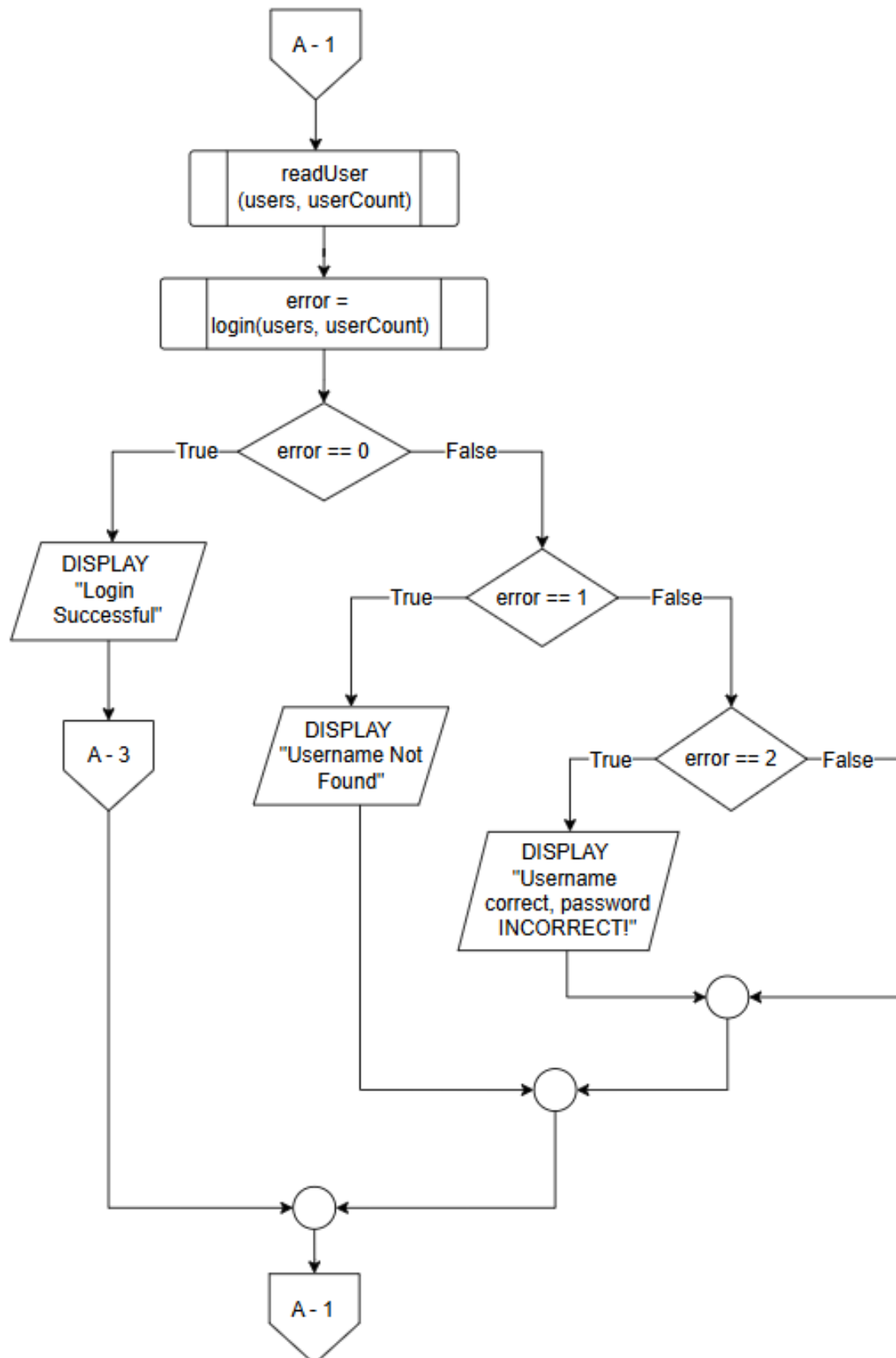
Close file.

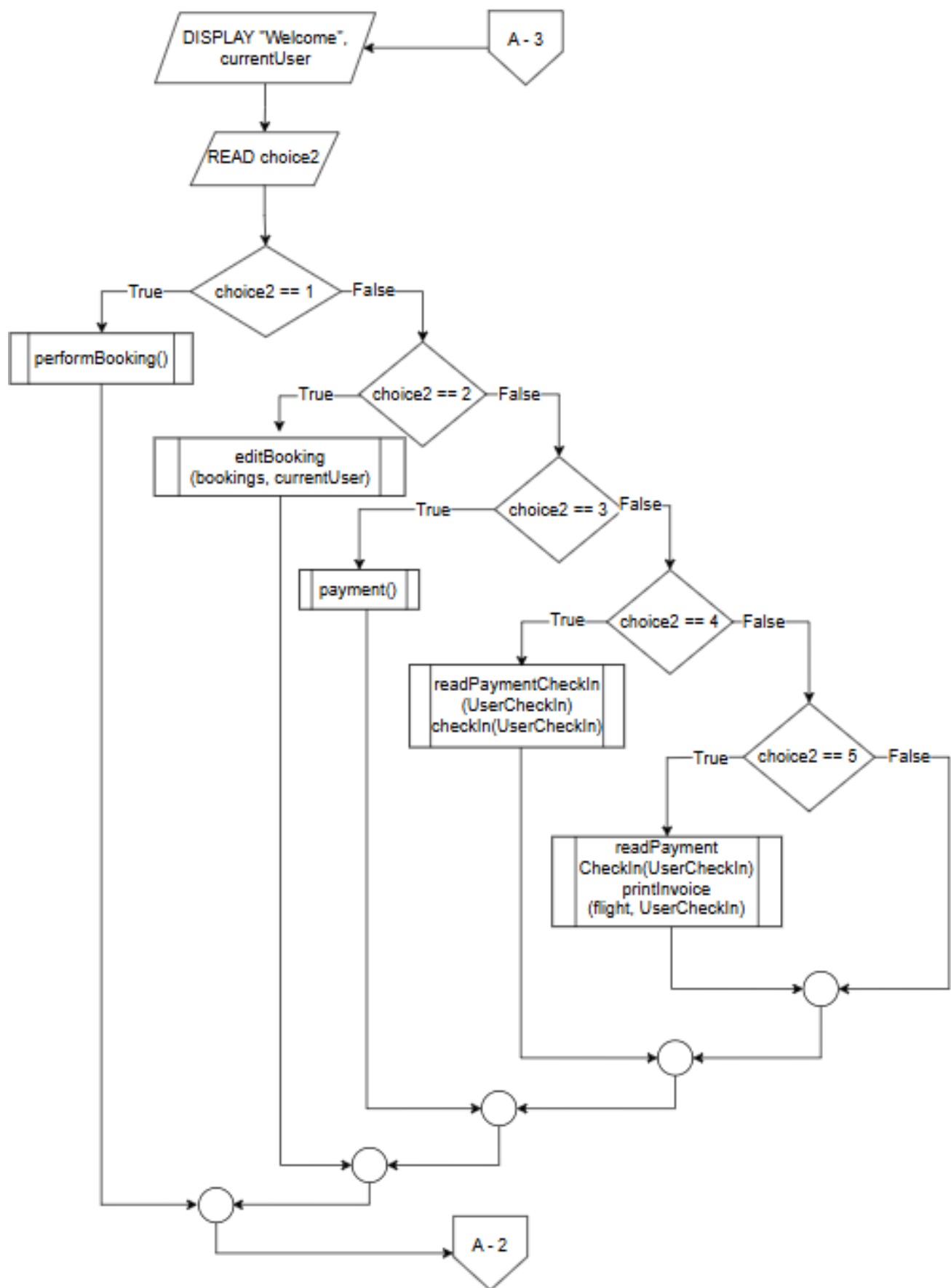
END

C. Flowchart

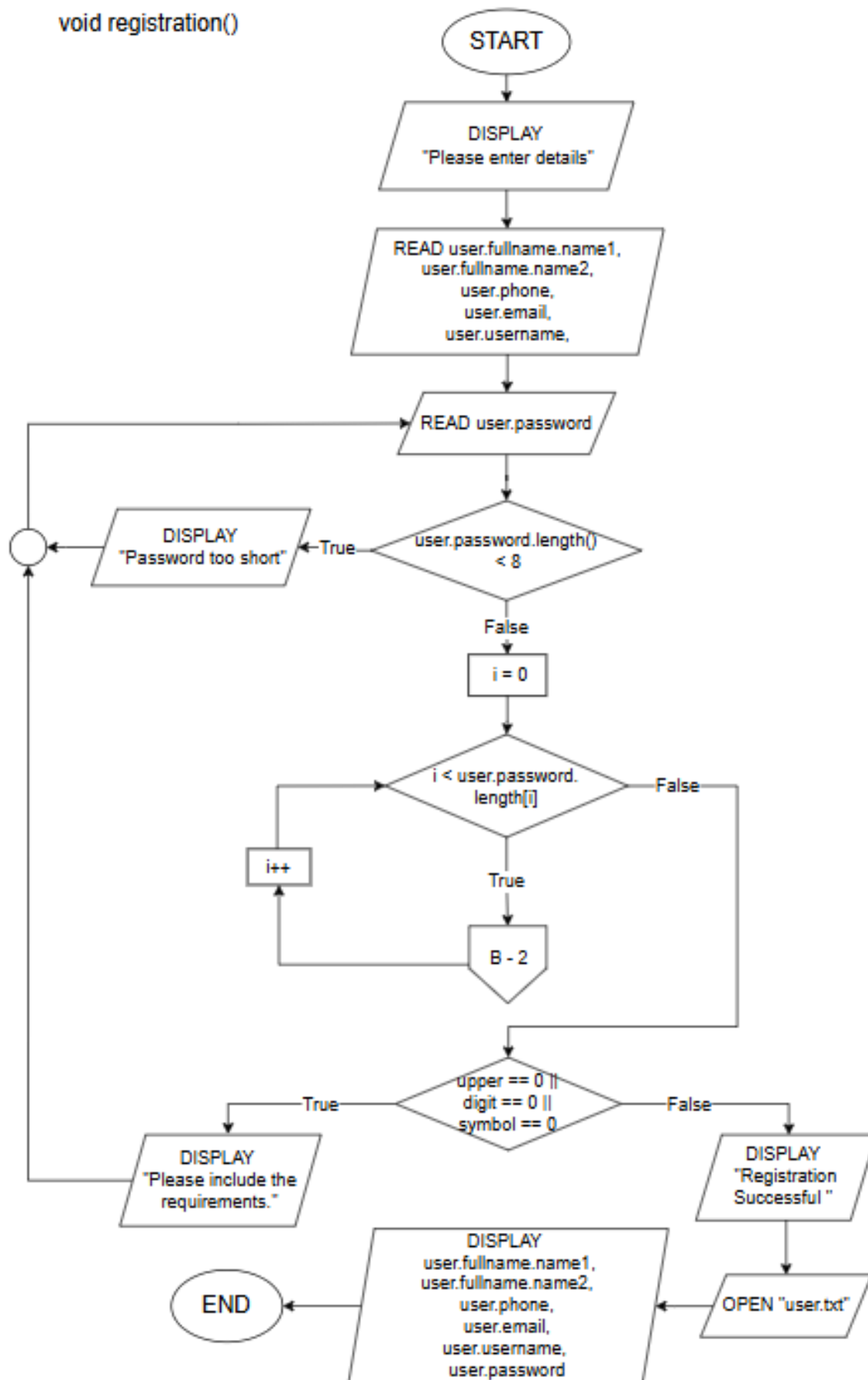
main()

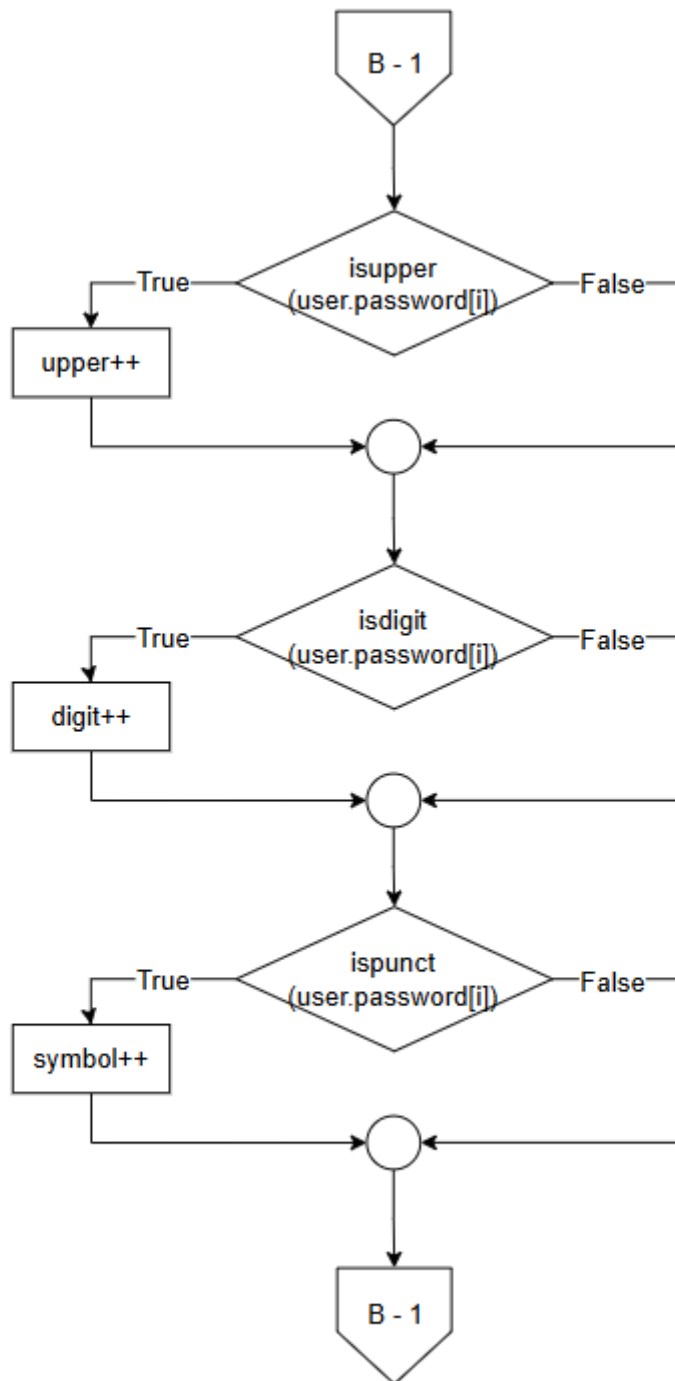




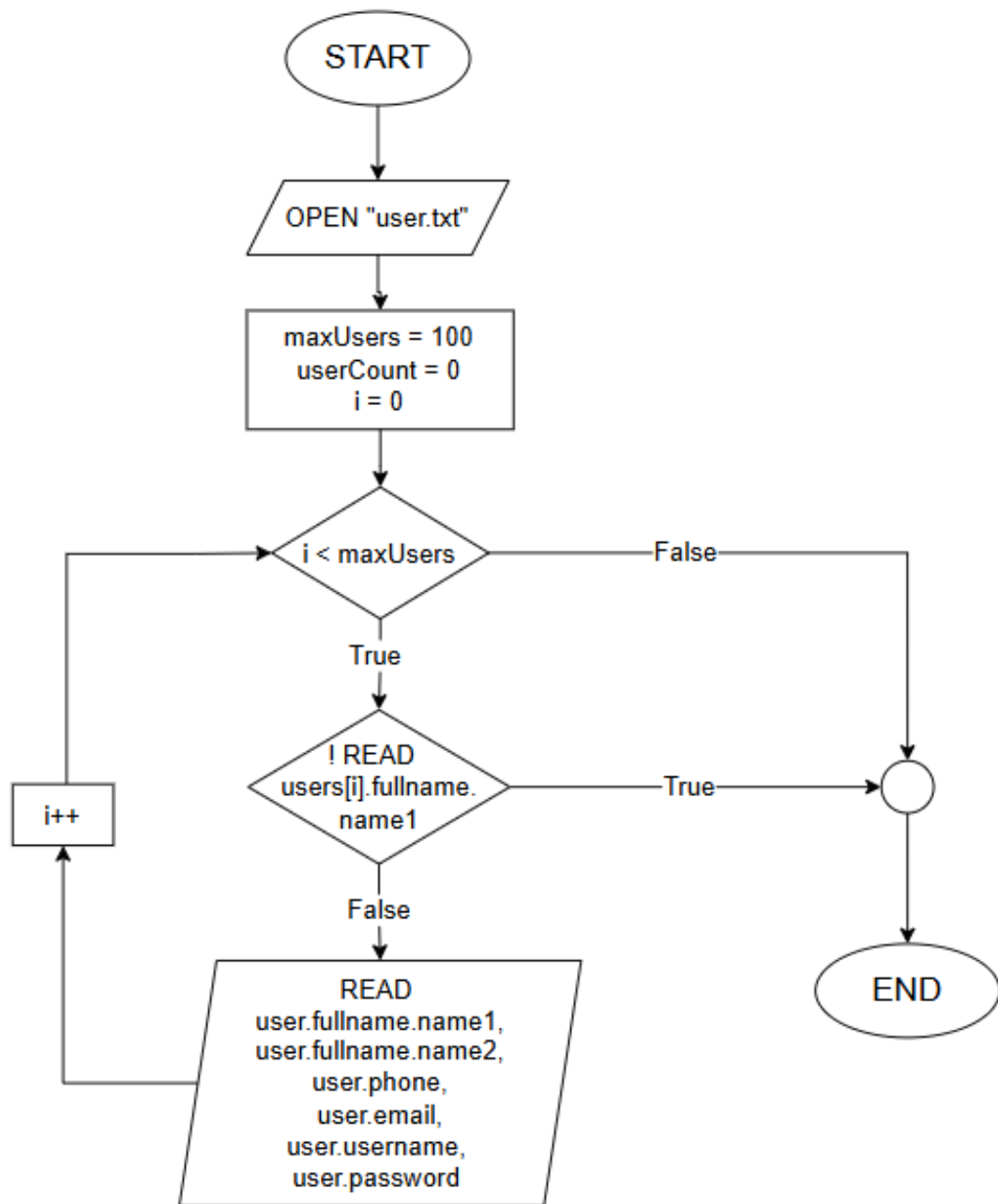


void registration()

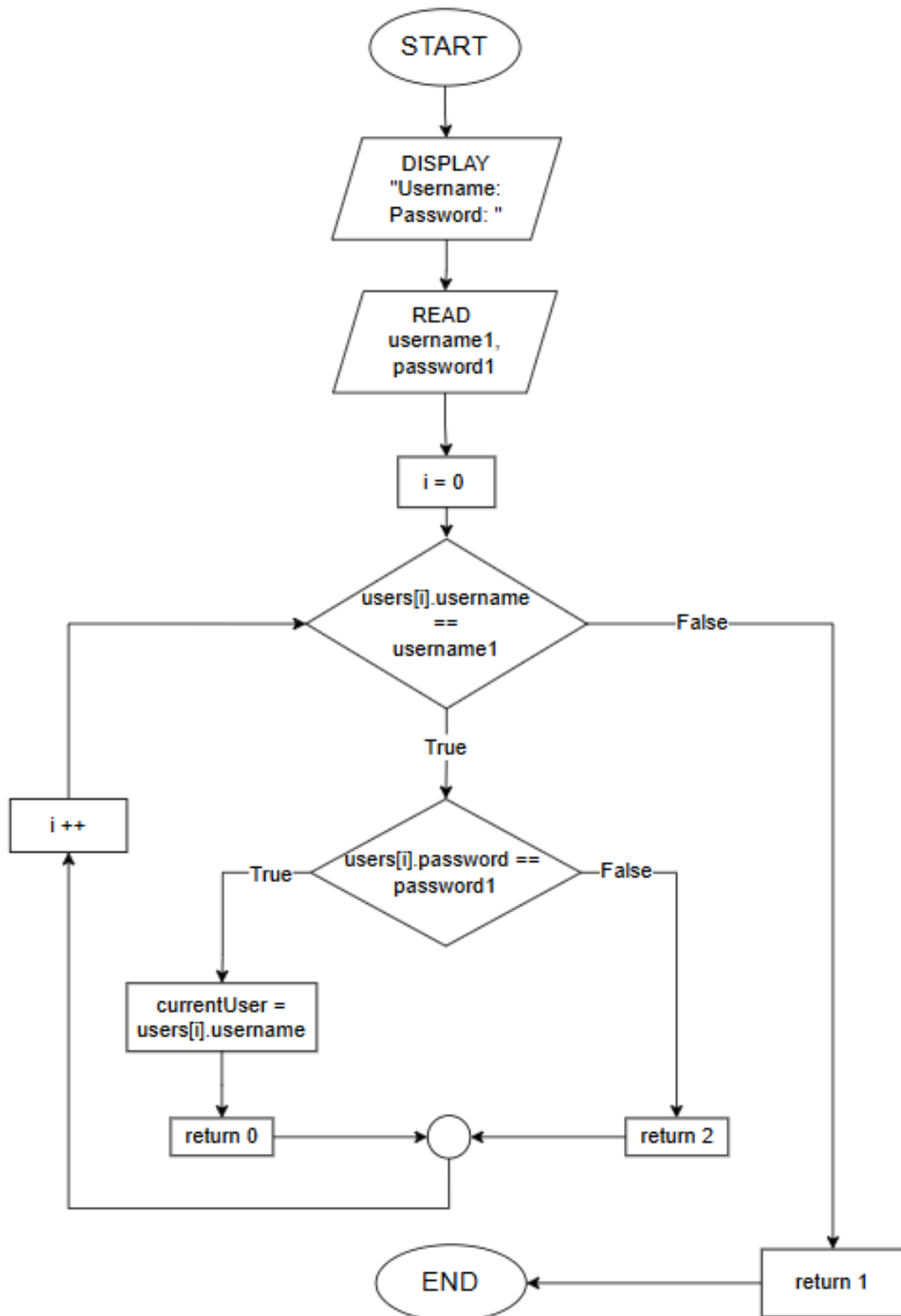




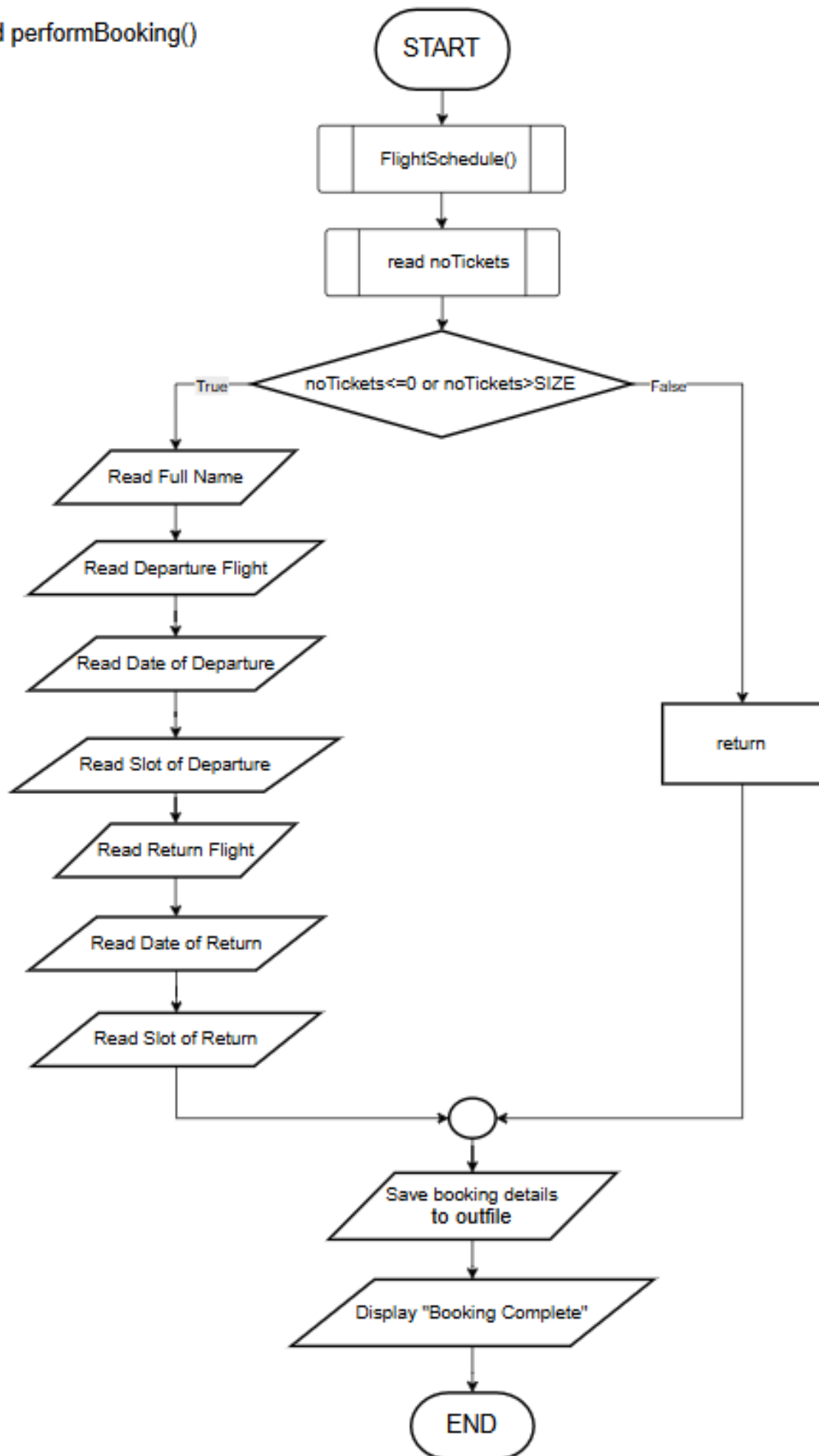
void readUser(info users[], int& userCount)



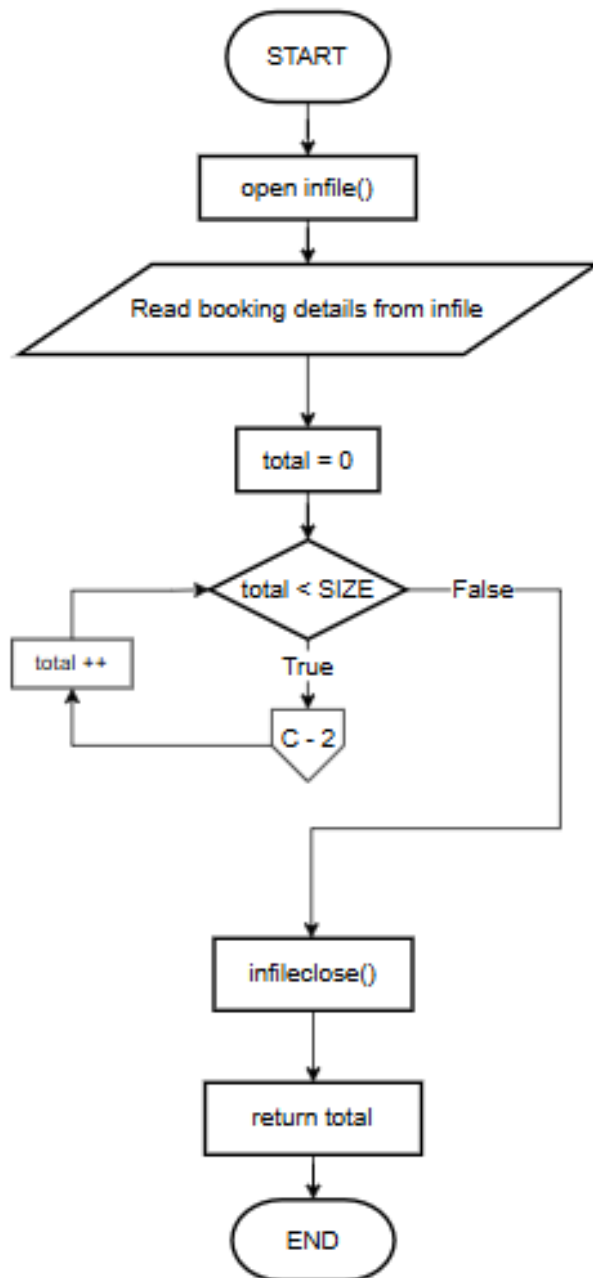
int login(info users[], int userCount)

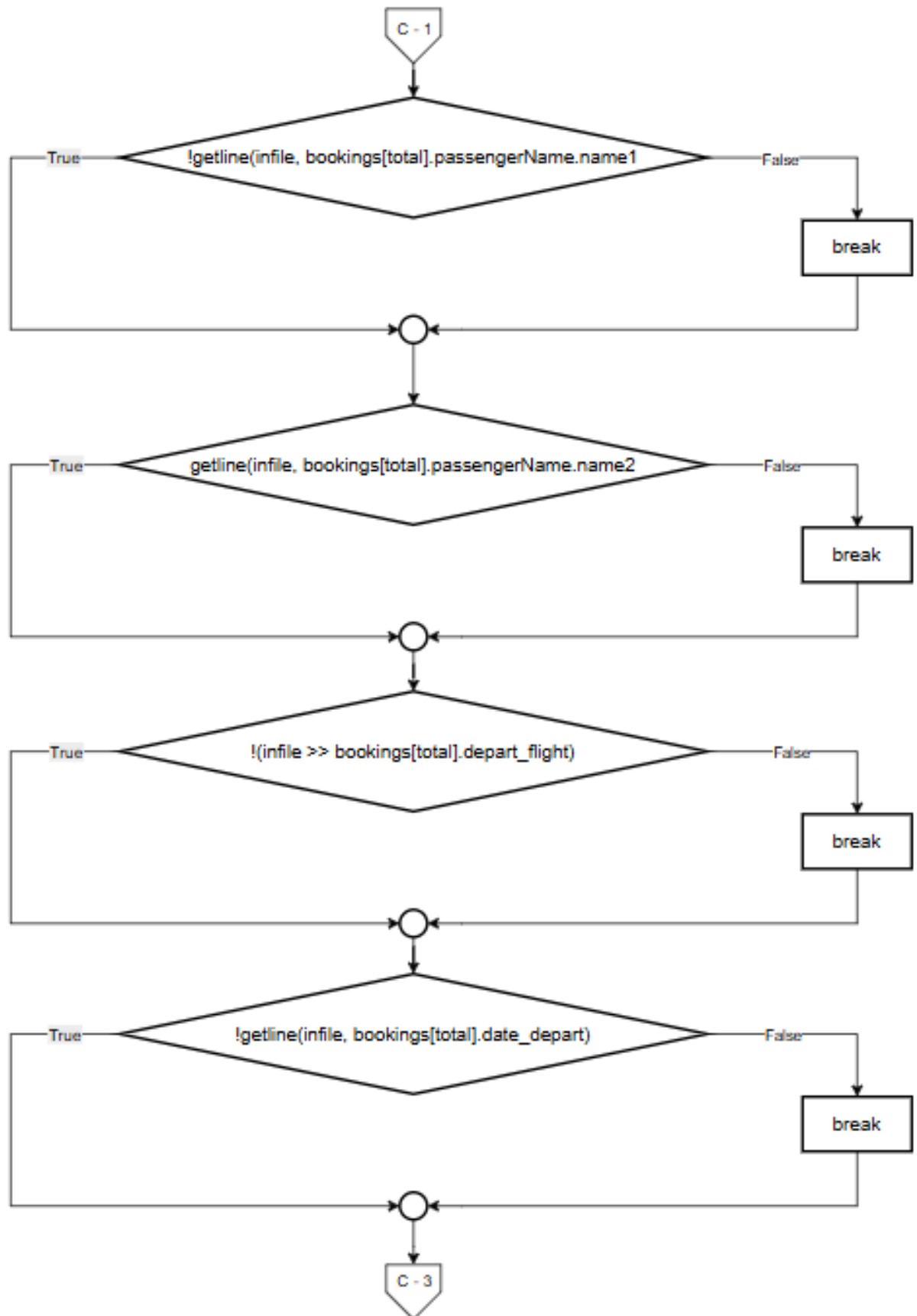


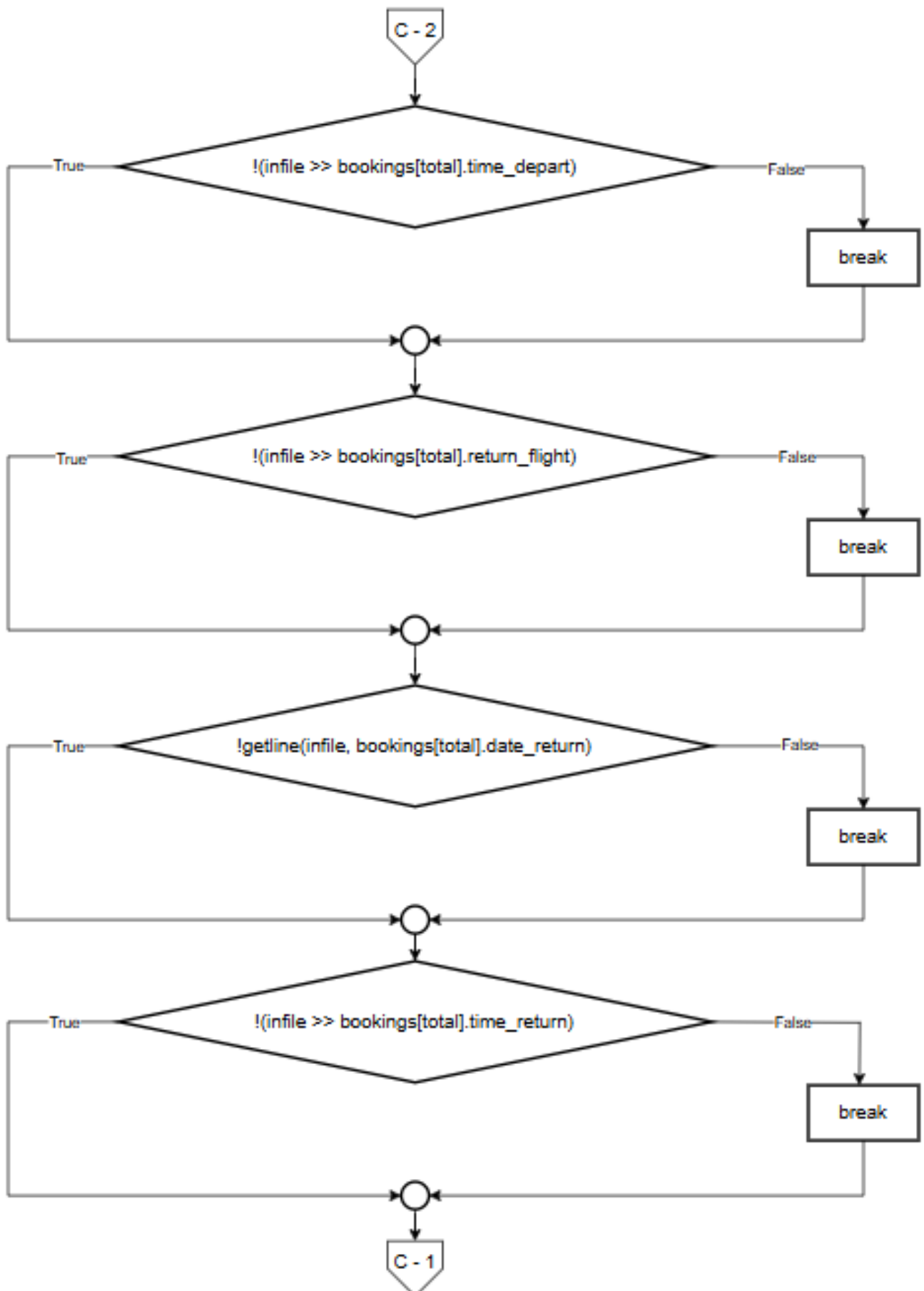
void performBooking()



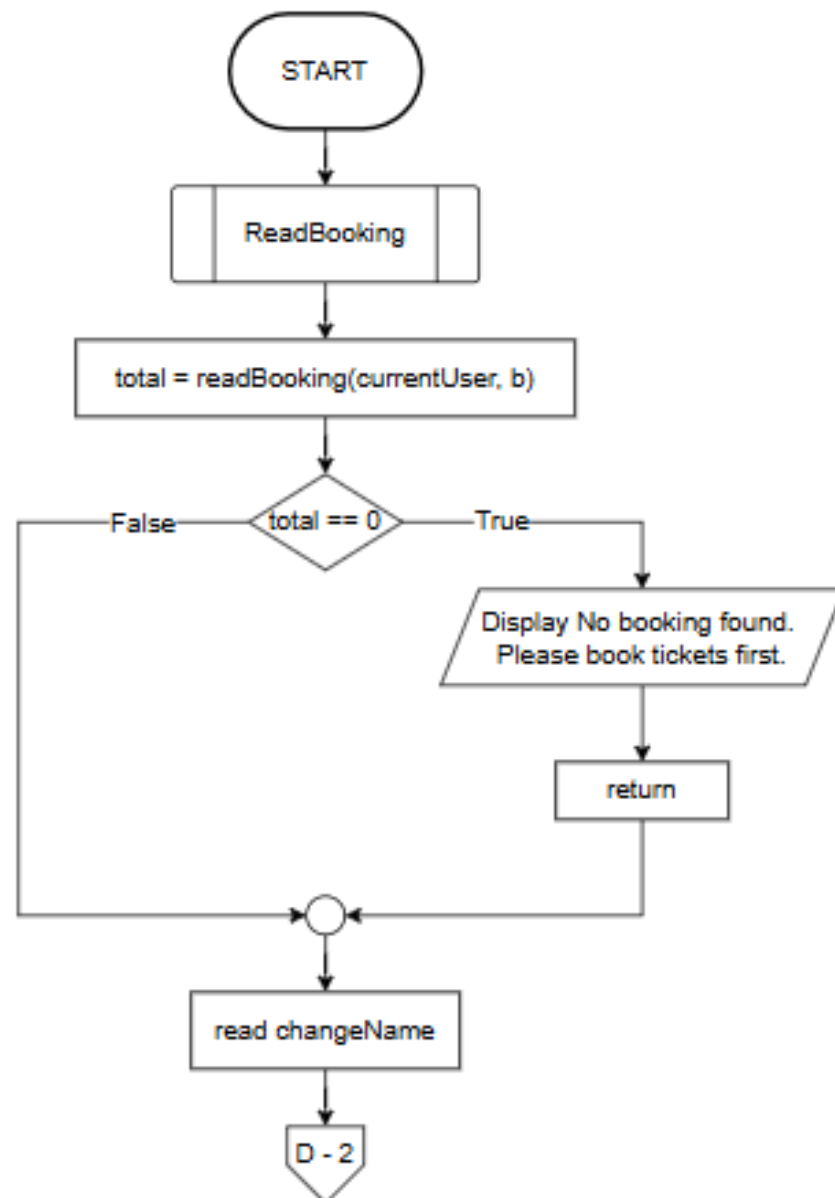
```
int readBooking(string,booking[])
```

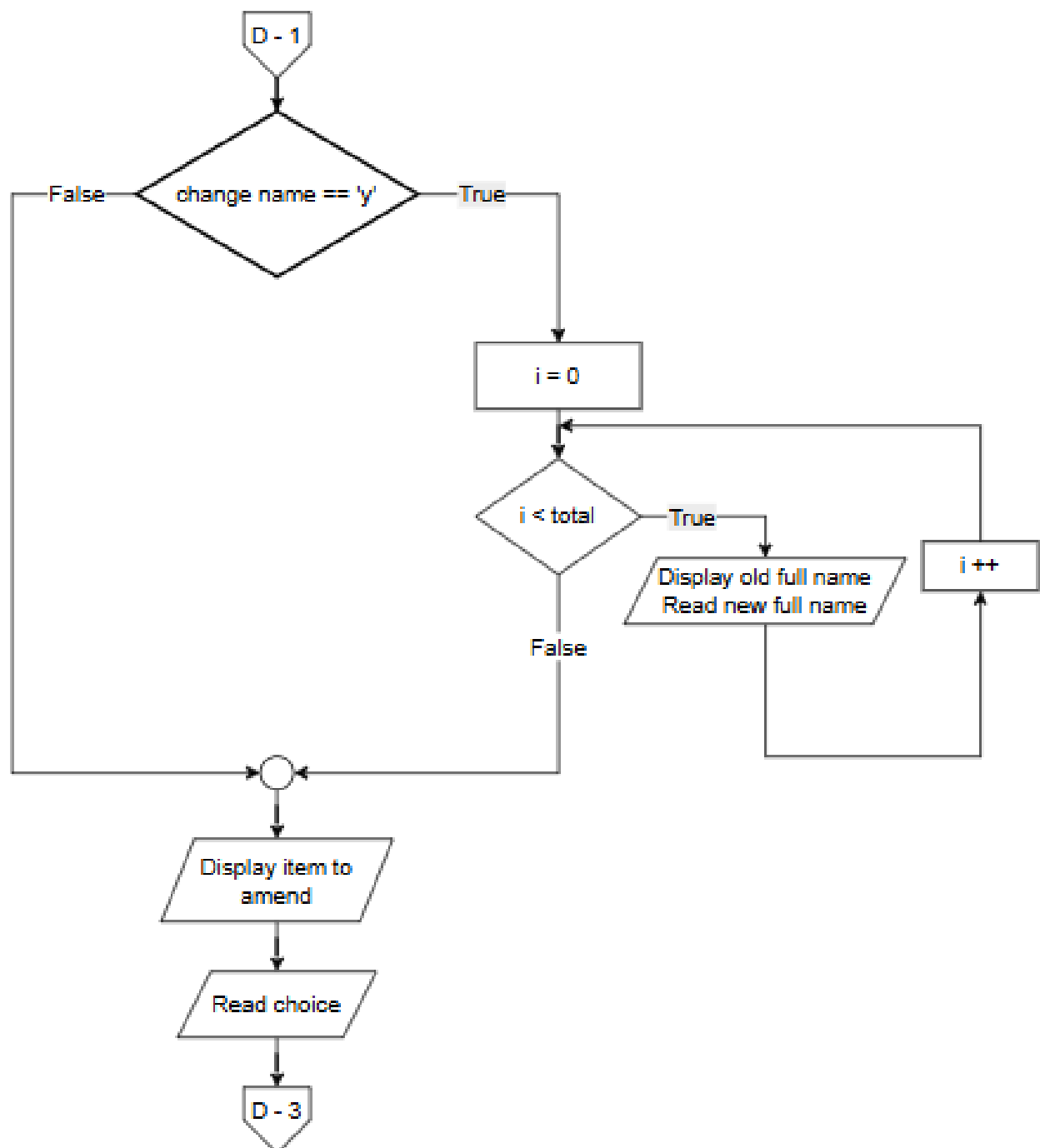


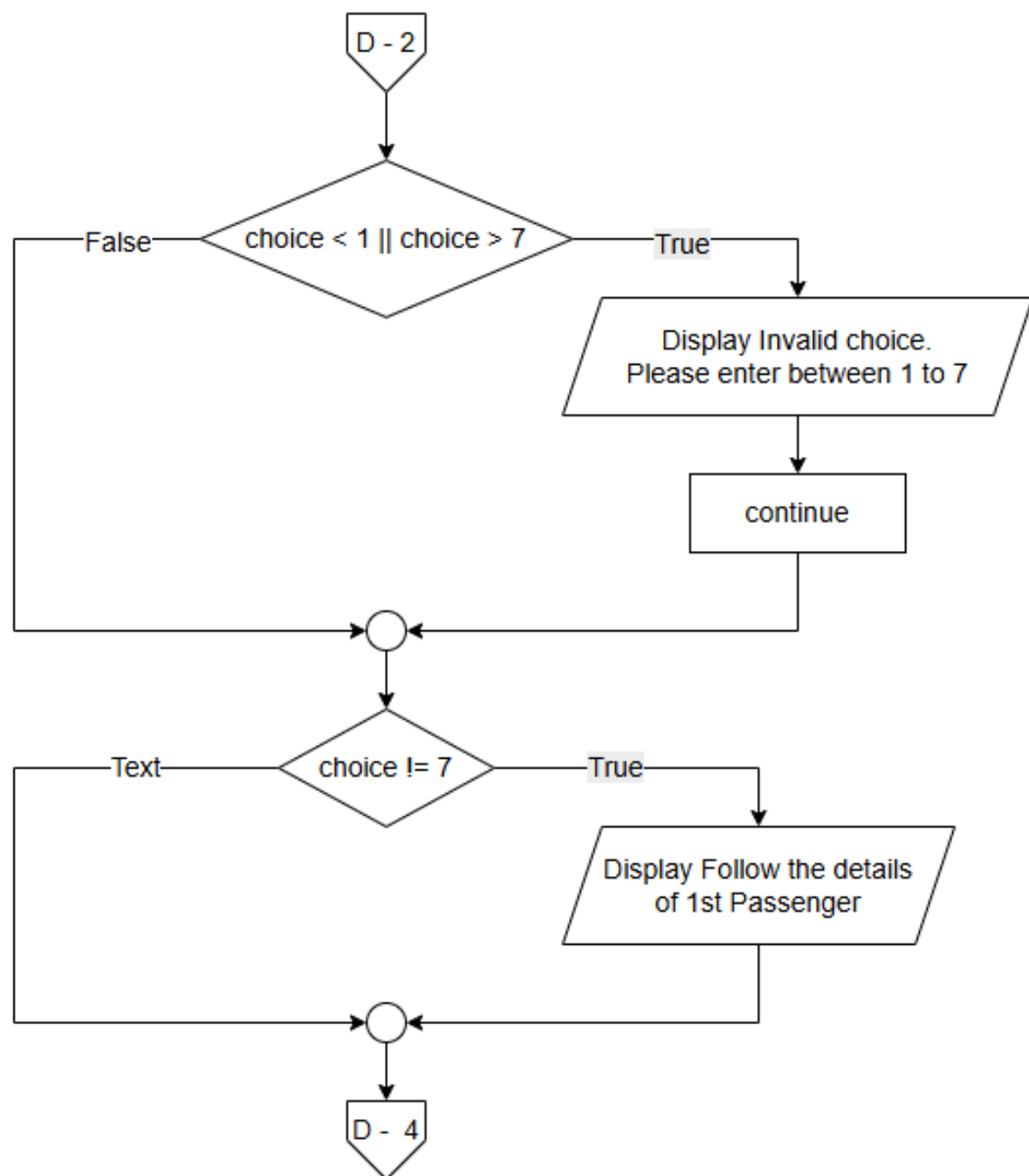


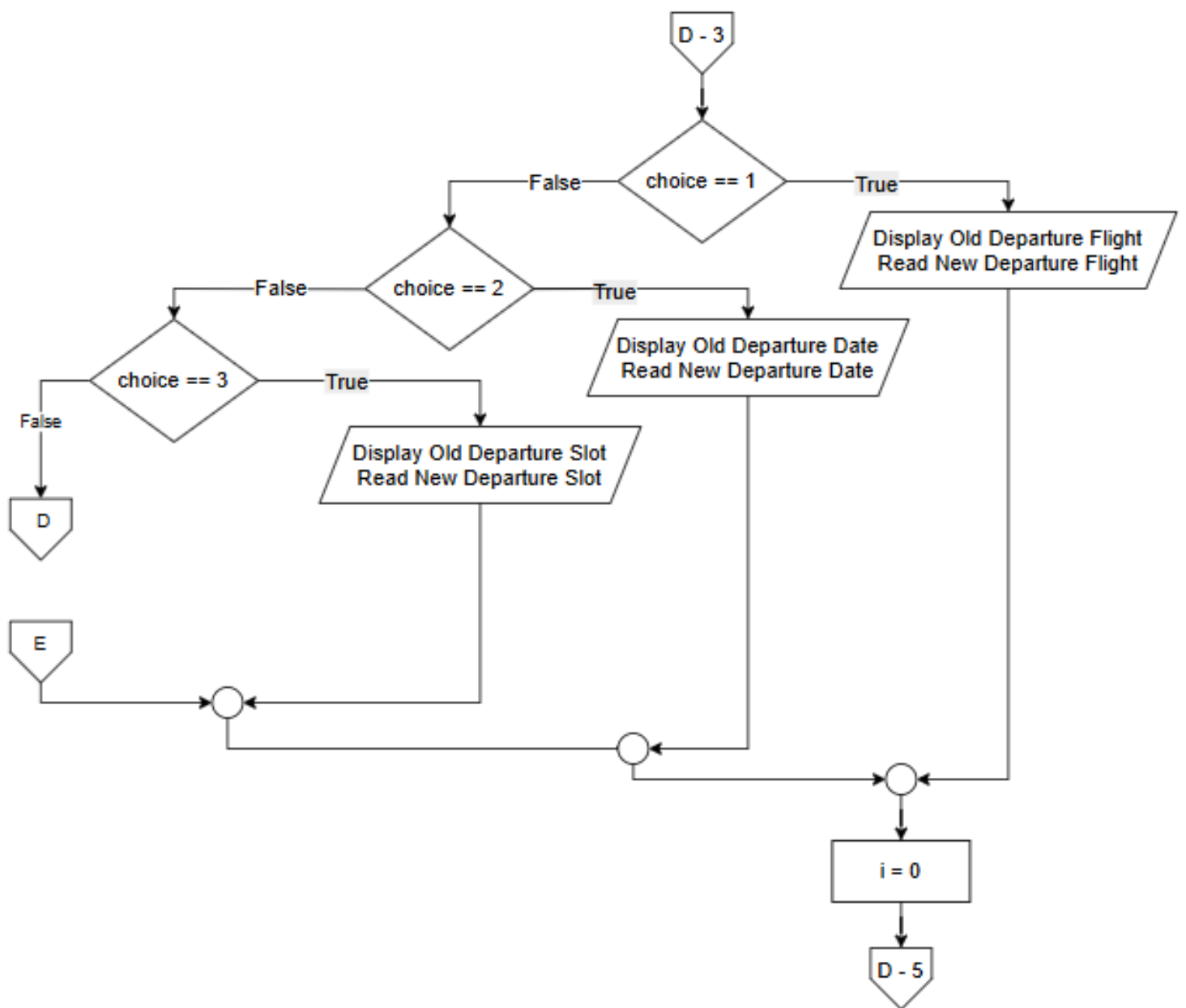


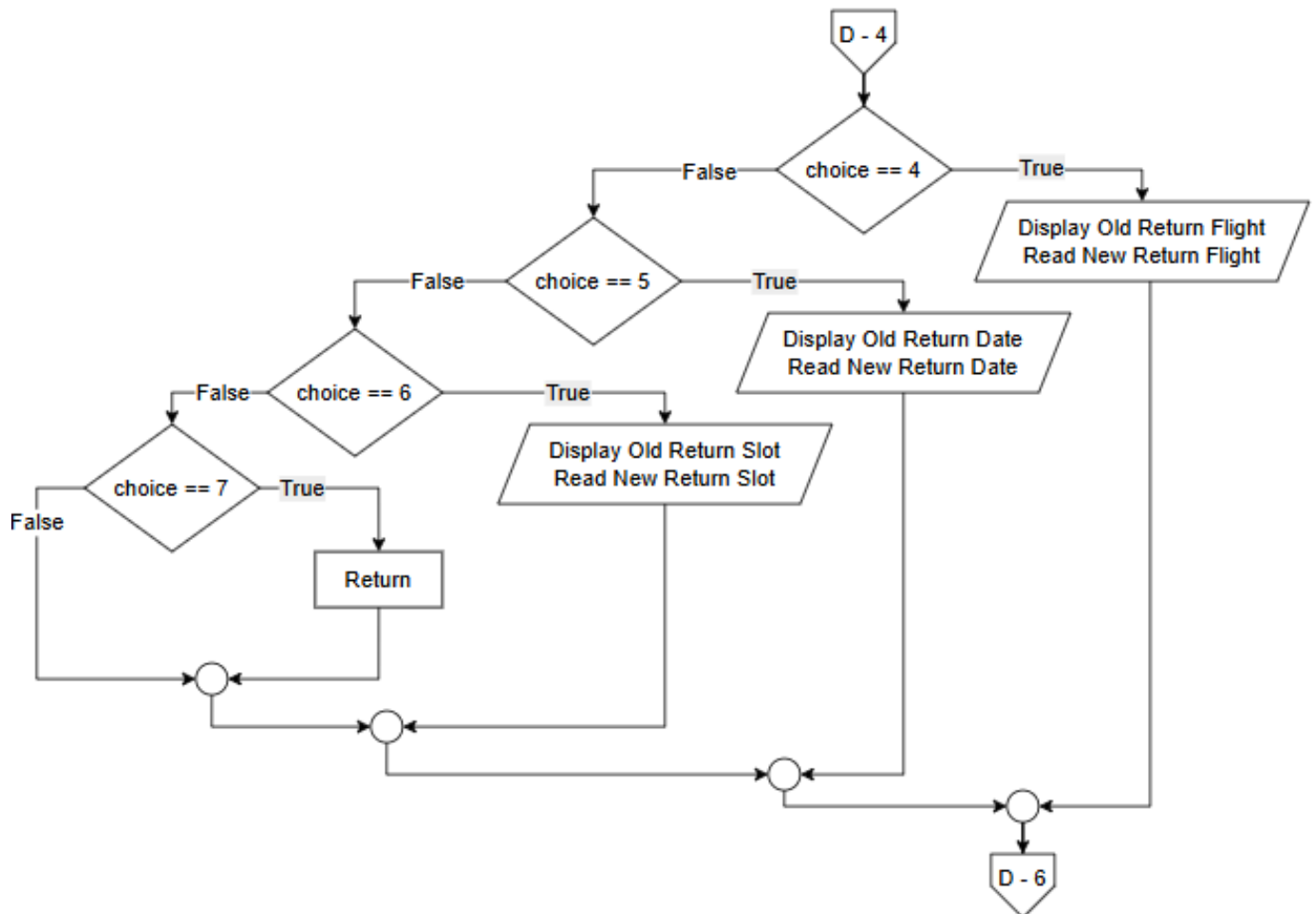
void editBooking(booking[], string)

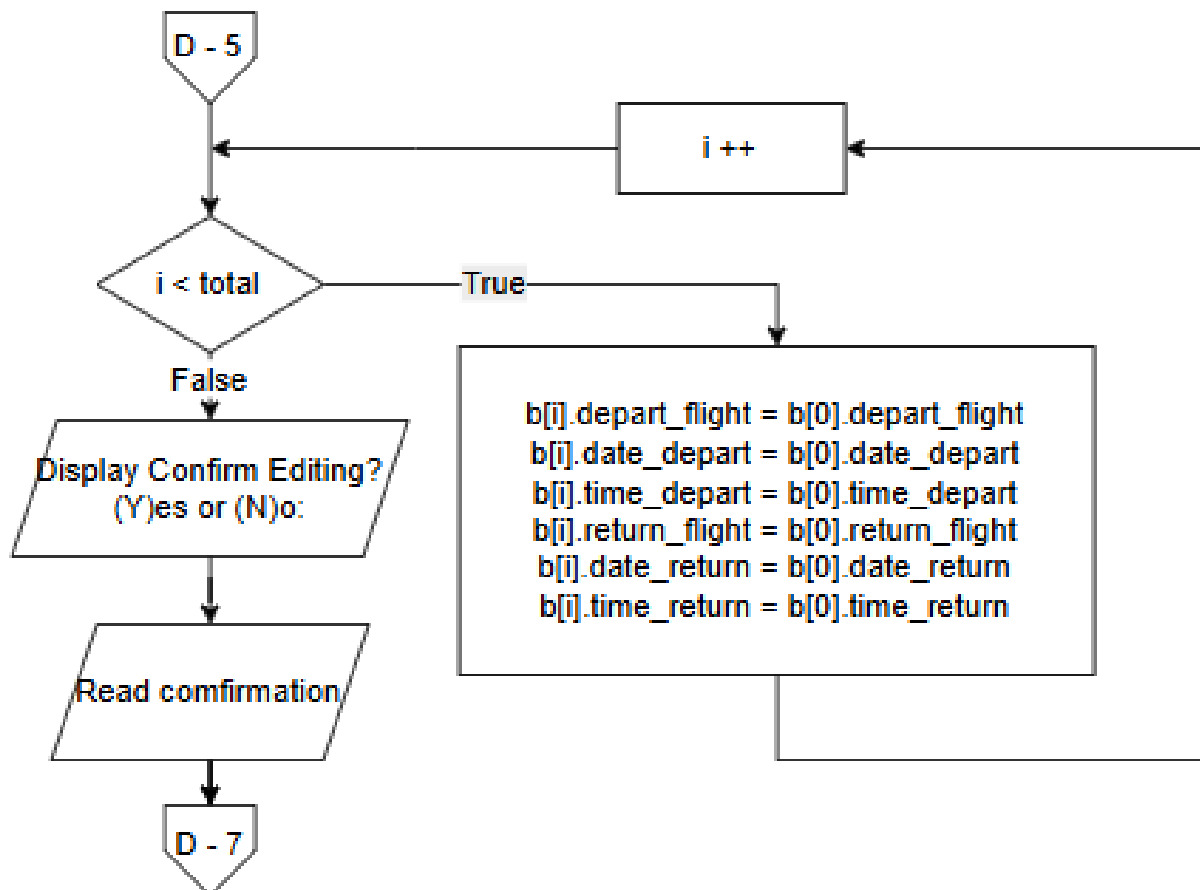


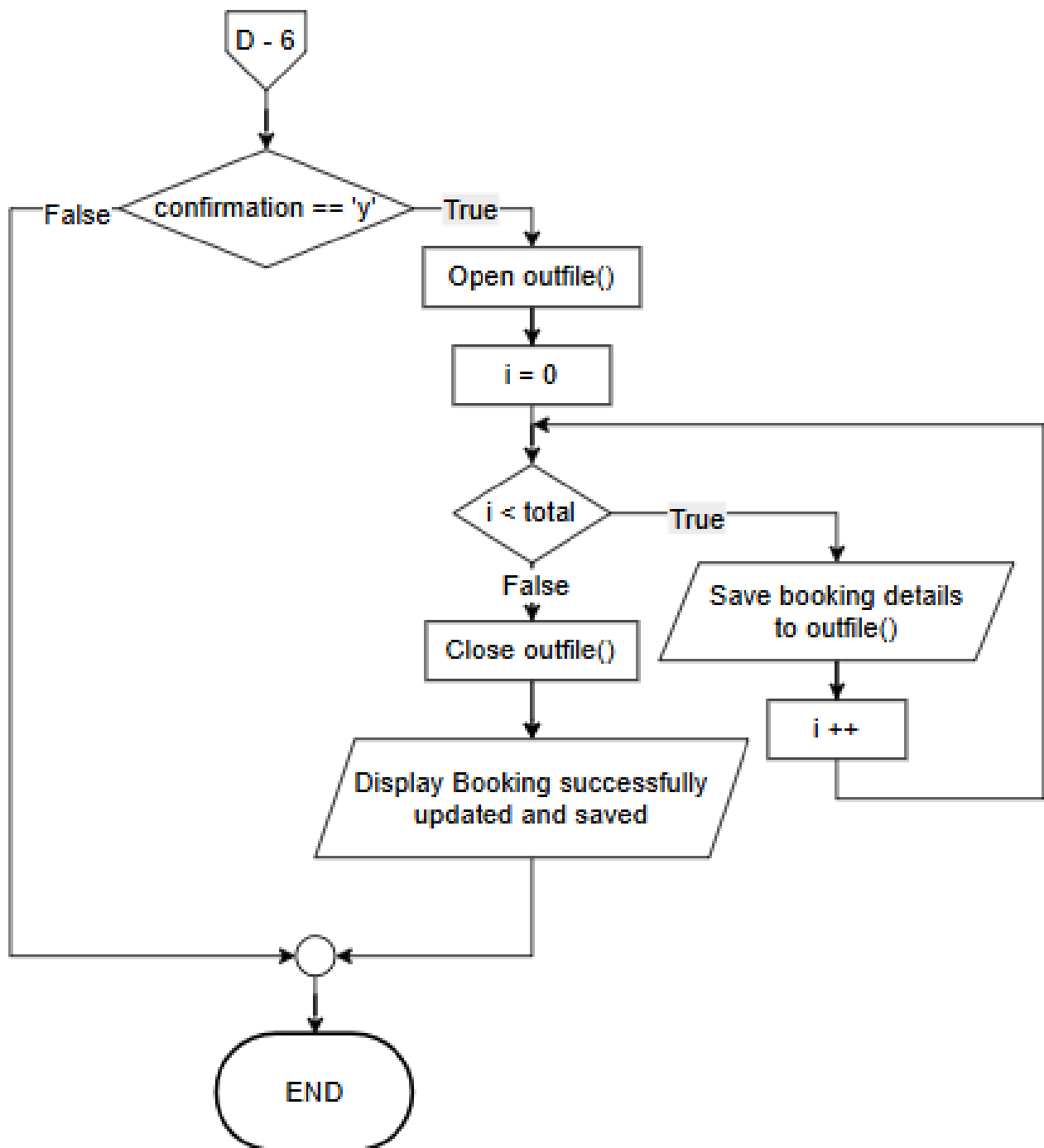




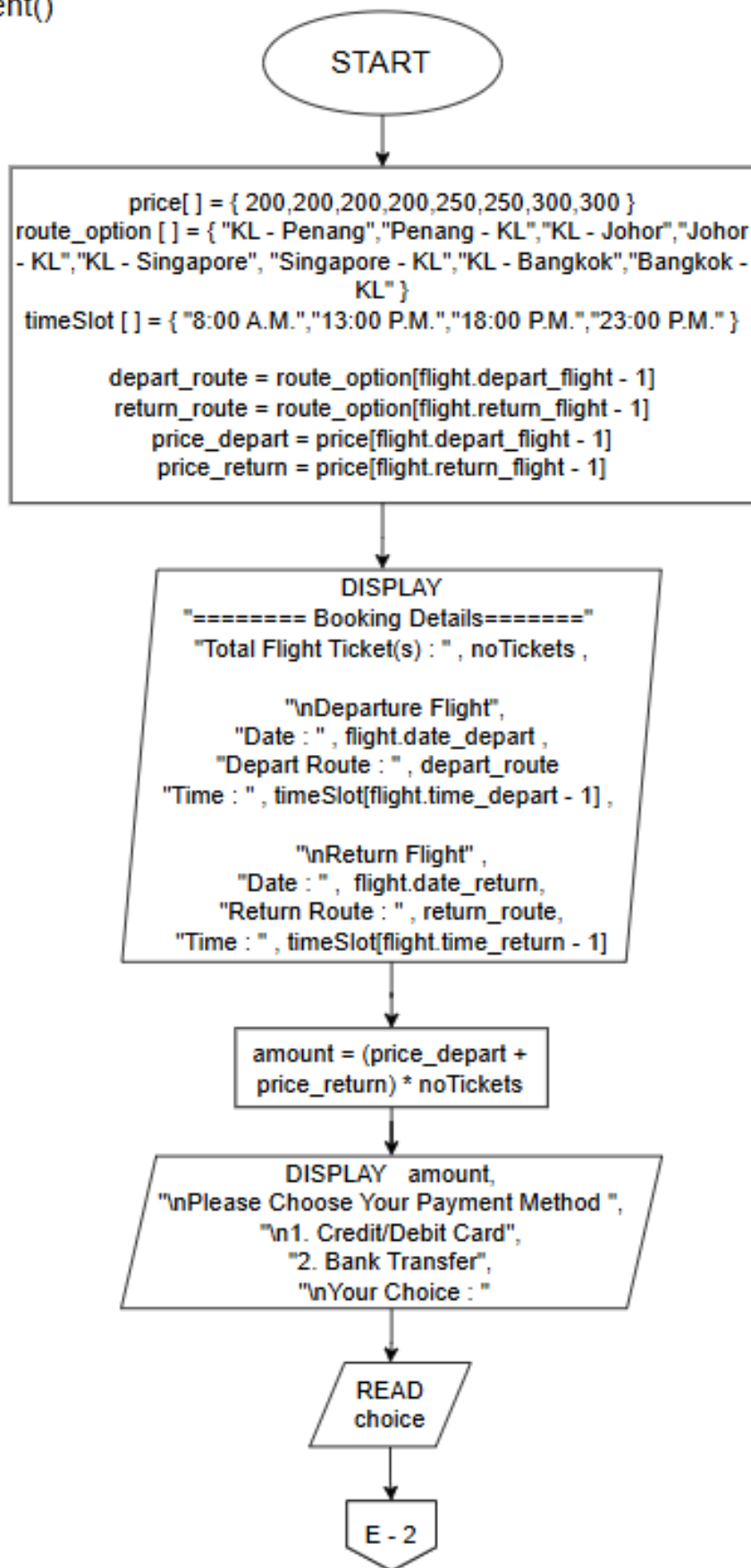


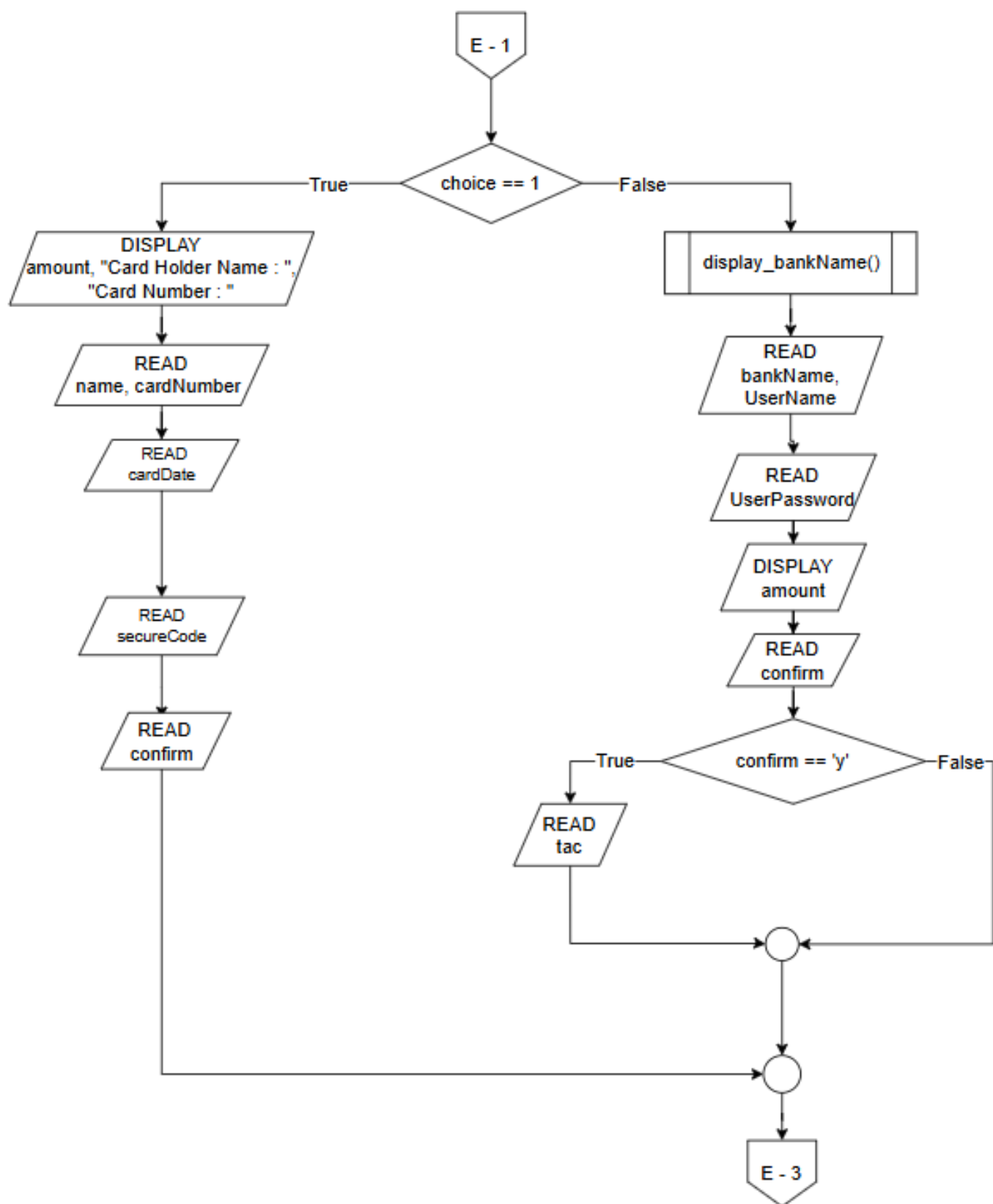


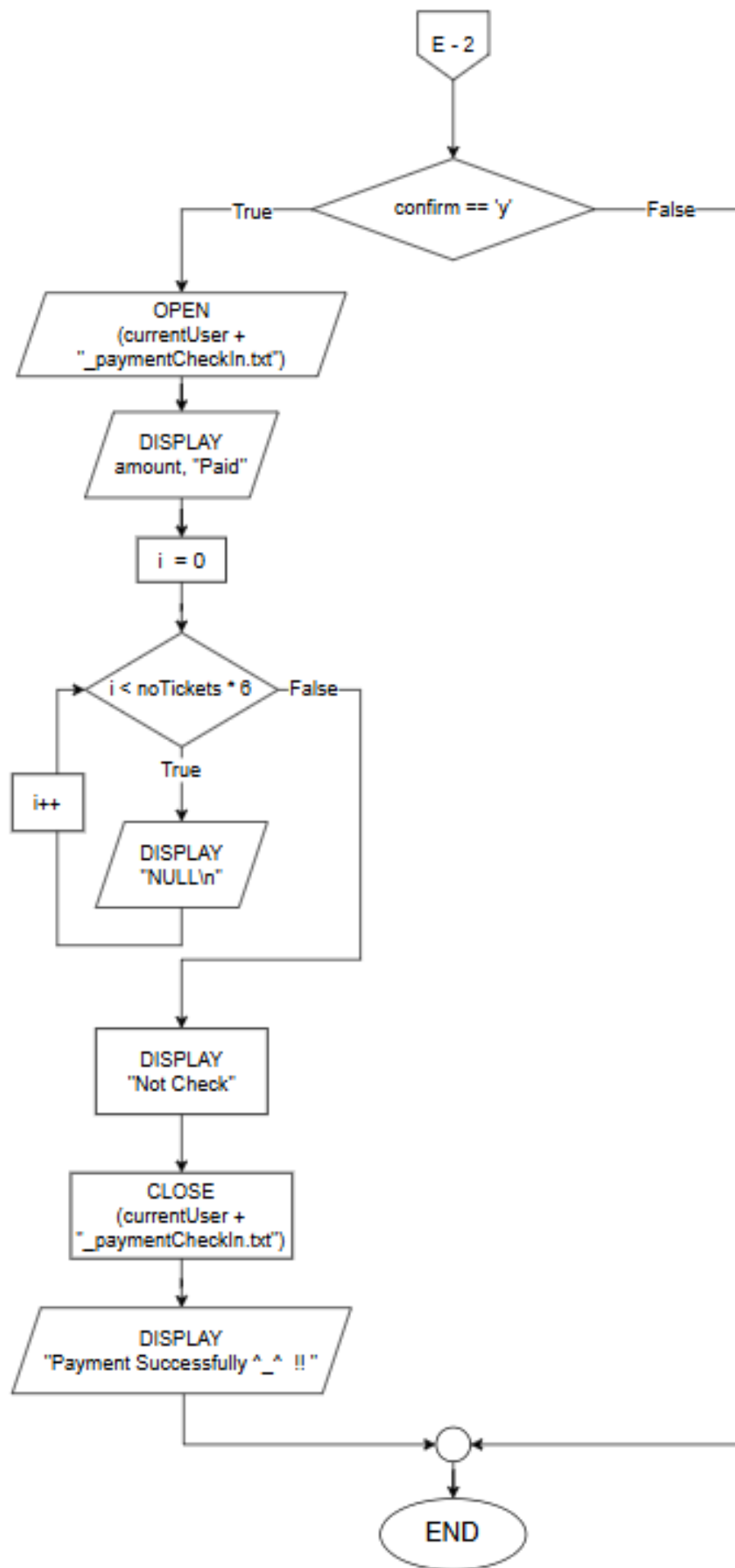




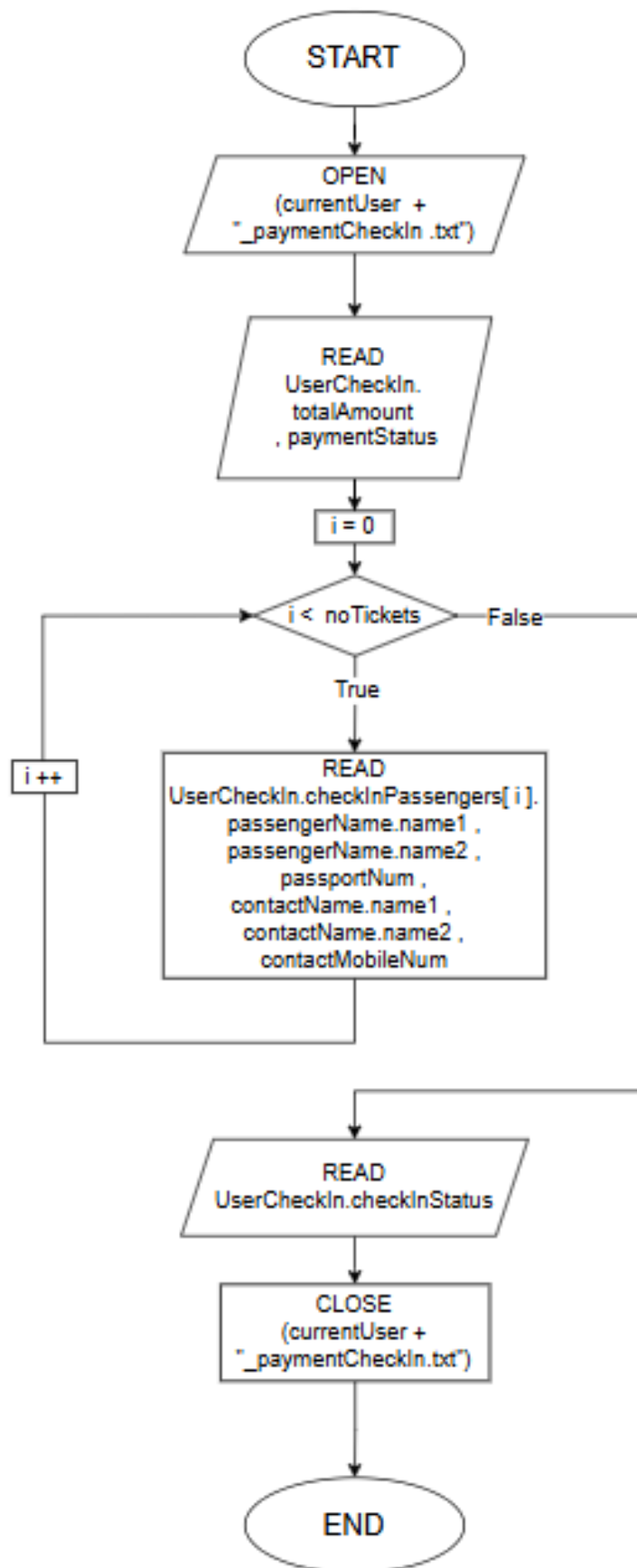
void payment()



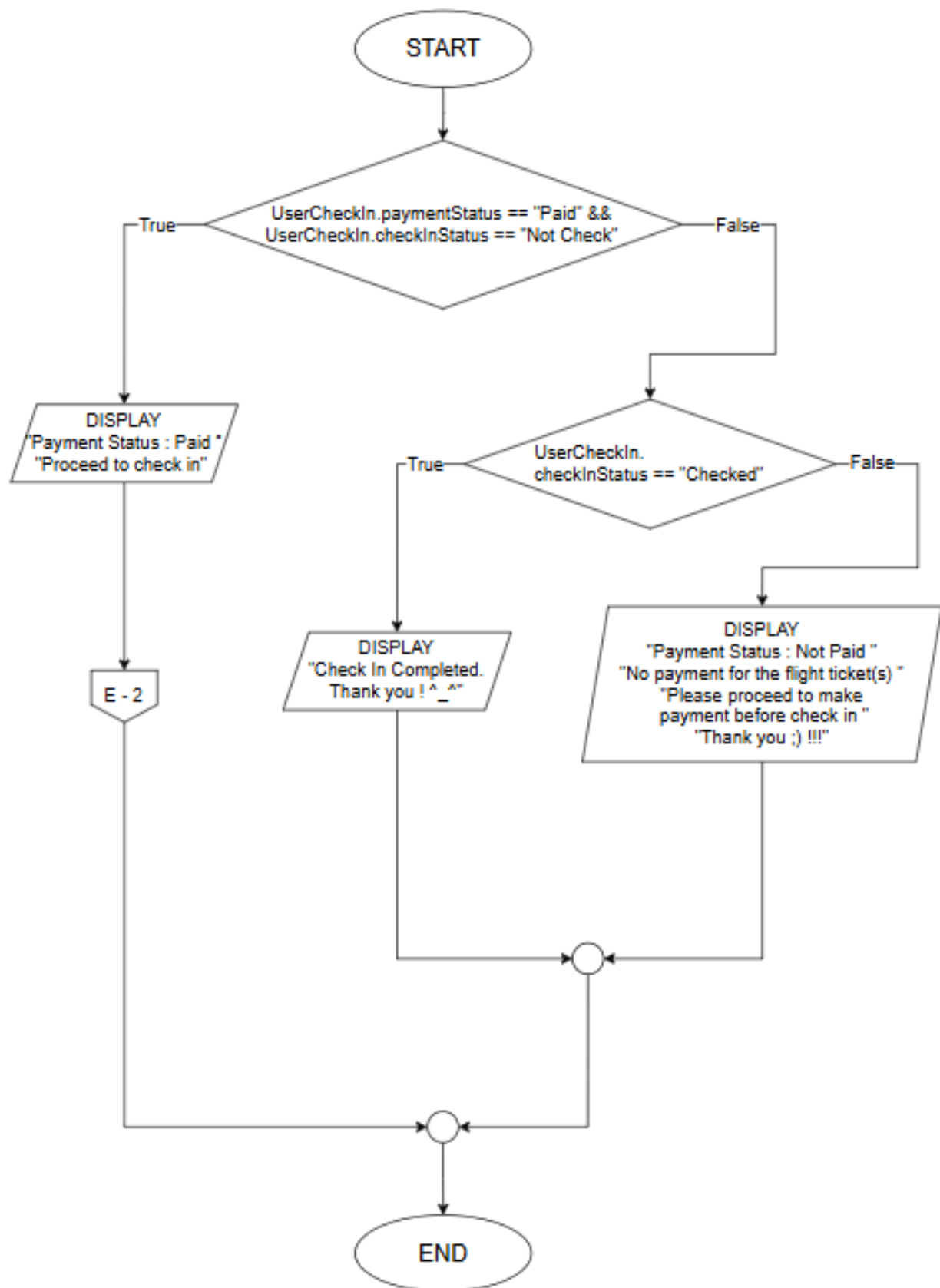


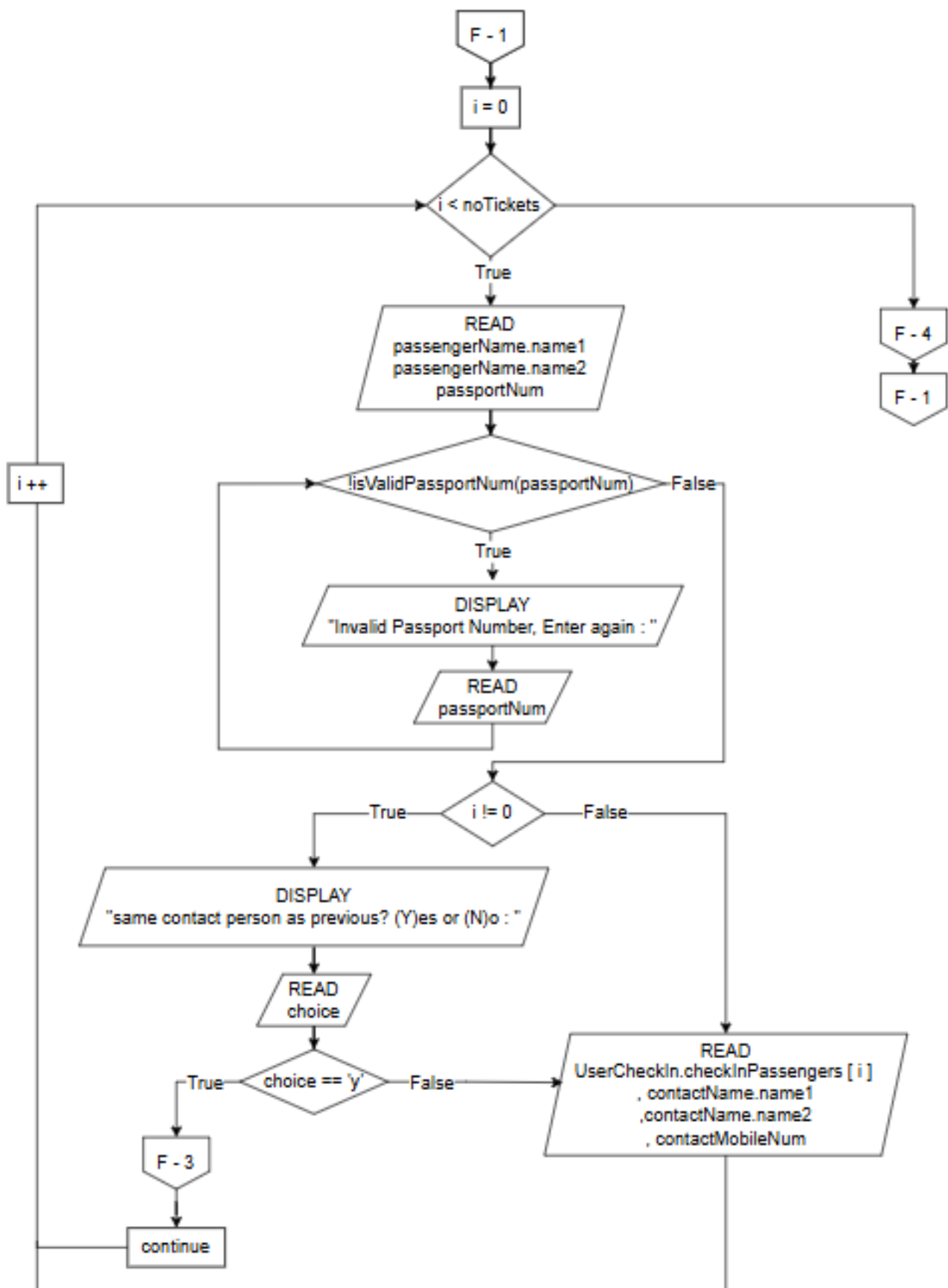


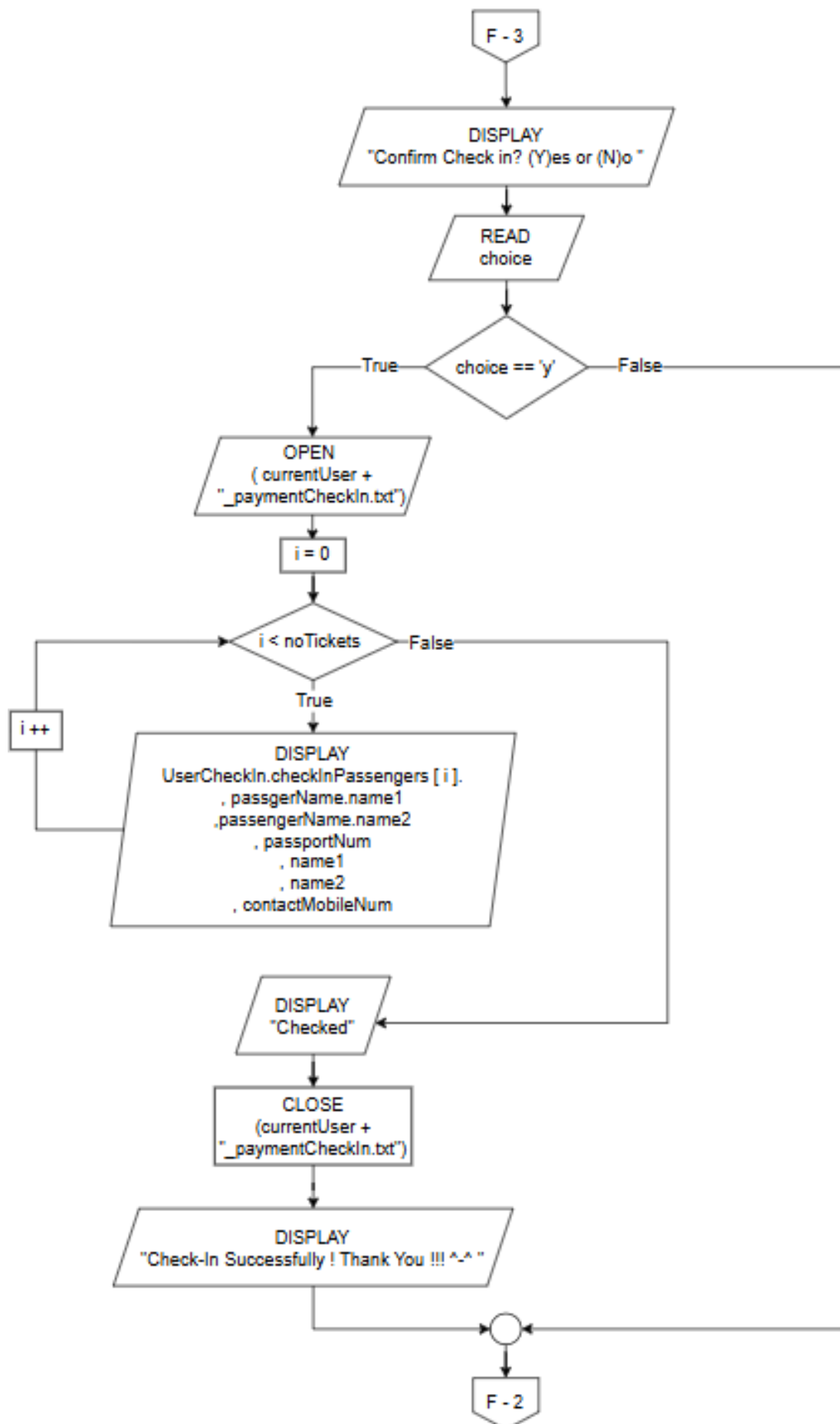
void readPaymentCheckIn (PaymentCheckInStatus& UserCheckIn)



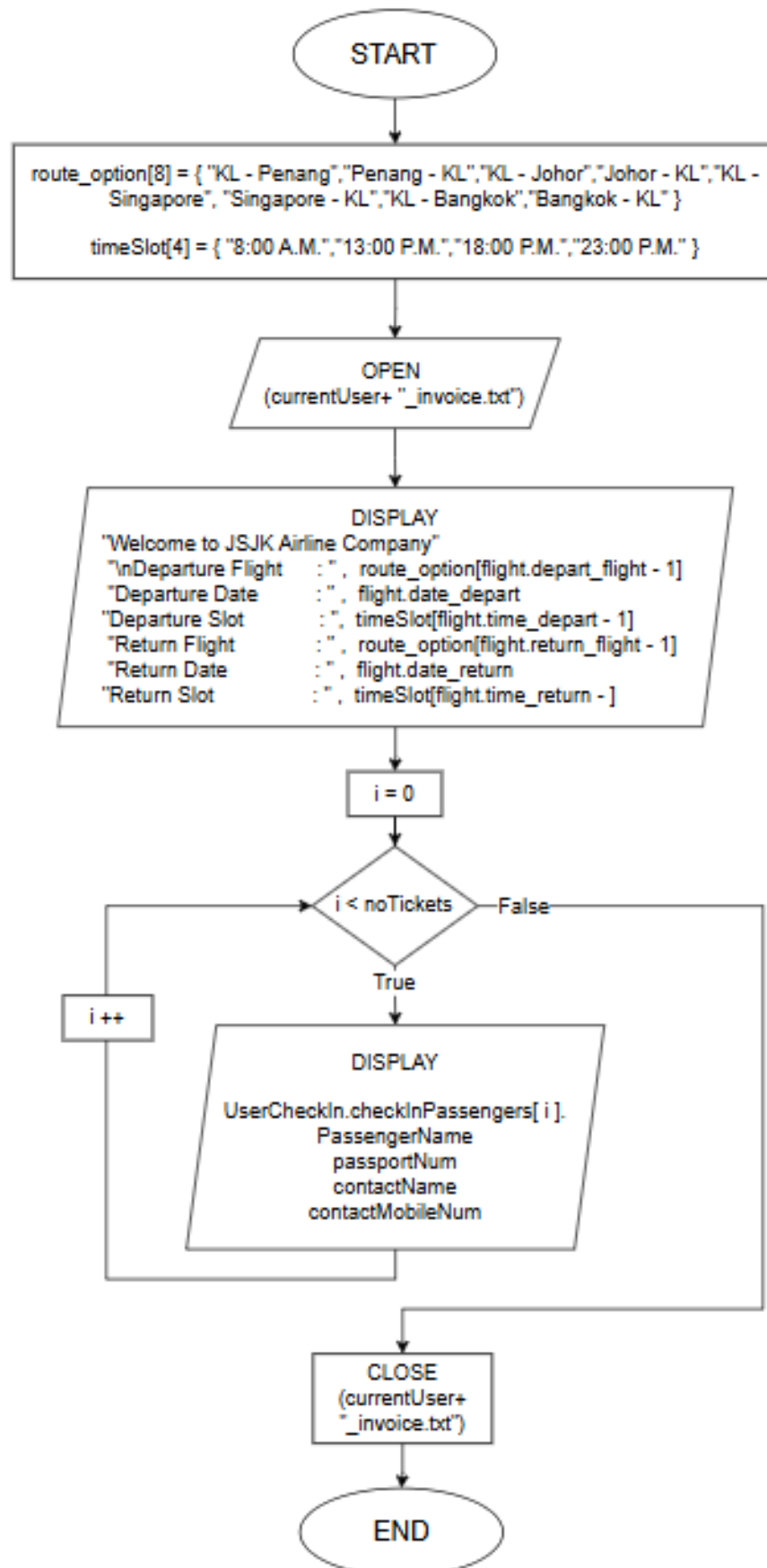
checkIn(paymentCheckInStatus& UserCheckIn)







void printInvoice (booking flight, PaymentCheckInStatus UserCheckIn)



D. Test Cases

Test Case 1: Input Validation for registration

```
===== Register =====
Please enter these details to register!!!

First Name      : Justin
Last Name       : Drew Bieber
Mobile Number   : 0125201234
Email           : justin@gmail.com
Username        : justinB

Password (Must include these requirements)
A) 8 Characters
B) 1 Symbol
C) 1 Uppercase
D) 1 Lowercase

Password: abc
Password too short. Must be at least 8 characters.

Password: abc12345
Please include one Uppercase & one Symbol.

Password: ABC12345
Please include one Symbol.

Password: ABC12345@

Registration Successfull !!
```

Test Case 2: Password Validation for login

```
===== Login =====
Username : justinb
Password : ABC12345@

Username NOT FOUND!
If you have not registered yet, please register first o.0 !!!
===== Login =====
Username : justinB
Password : ABC12345
Username correct, password INCORRECT!

===== Login =====
Username : justinB
Password : ABC12345@

Login Successful !!!
```

Test Case 3: Flight Booking

```
===== Flight Available =====  
1. KL - Penang      --> RM200  
2. Penang - KL      --> RM200  
3. KL - Johor       --> RM200  
4. Johor - KL       --> RM200  
5. KL - Singapore   --> RM250  
6. Singapore - KL   --> RM250  
7. KL - Bangkok     --> RM300  
8. Bangkok - KL     --> RM300  
  
Available Departure/Return Time Slots:  
1. 08:00 A.M.  
2. 13:00 P.M.  
3. 18:00 P.M.  
4. 23:00 P.M.  
  
Number of passenger(s): 2  
  
Please provide the details as below:  
  
Passenger 1  
First Name          : Justin  
Last Name           : Drew Bieber  
  
Departure Flight    : 1  
Departure Date (DD/MM/YYYY) : 01/01/2025  
Departure Time Slot : 1  
  
Return Flight       : 2  
Return Date (DD/MM/YYYY) : 05/01/2025  
Return Time Slot    : 2  
  
Passenger 2  
First Name          : Taylor  
Last Name           : Swift  
  
Departure Flight    : 1  
Departure Date (DD/MM/YYYY) : 01/01/2025  
Departure Time Slot : 1  
  
Return Flight       : 2  
Return Date (DD/MM/YYYY) : 05/01/2025  
Return Time Slot    : 2  
  
Booking Completed !!!
```

Test Case 4: Edit Passenger Name & Flight Details

```
===== Edit Booking =====
Change passenger name? (Y)es or (N)o: y

Passenger 1 current name: Justin Drew Bieber
New First Name : Justin
New Last Name  : Bieber

Passenger 2 current name: Taylor Swift
New First Name : Taylor
New Last Name  : Alison Swift
Passenger name changed successfully !!!

===== Edit Booking =====

Select item to amend:
1. Departure Flight
2. Departure Date
3. Departure Slot
4. Return Flight
5. Return Date
6. Return Slot
7. Exit

Select: 1

Follow the details of 1st Passenger (Justin Bieber)
Old Departure Flight: 1
New Departure Flight: 3

Confirm Editing? (Y)es or (N)o: y
Booking successfully updated and saved!
```

Test Case 5: Payment

Credit / Debit Card

```
===== Payment =====
Total Amount      : RM800

Card Holder Name  : Justin
Card Number       : 50030

Card Valid Date (MM/YYYY) : 03/2027
Security Code    : 123

Confirm Transaction? (Y)es or (N)o: y
Payment Sucessfully ^-^ !!
```

Bank Transfer

```
===== Payment =====
----- Bank Name -----
1. Maybank          5. CIMB Bank
2. Public Bank      6. OCBC Bank
3. AmBank           7. Alliance Bank
4. HSBC             8. Standard Chartered

Select Bank : 4

Please enter your bank details :0
Username      : John
Password      : Bieber
Payment       : RM800

Confirm and request TAC (Y)es or (N)o: y
Enter TAC    : 123
Payment Sucessfully ^-^ !!
```

Test Case 6: Check In

```
===== Check In =====  
  
Payment Status : Not Paid  
No payment for the flight ticket(s)  
Please proceed to make payment before check in  
Thank you :) !!!  
  
Press Enter to return to booking menu...
```

```
===== Check In =====  
  
Payment Status: Paid  
Proceed to check in ====>  
  
Passenger 1  
  
First Name      : Justin  
Last Name       : Bieber  
Passport No.    : A12345@  
  
Invalid Passport Number!  
Please include 1 uppercase & 5 digits only !!!  
  
Please enter again: A12345  
  
Contact Person First Name : Shawn  
Contact Person Last Name  : Mendes  
Contact Person Phone No.  : 0125201314  
  
Passenger 2  
  
First Name      : Taylor  
Last Name       : Alison Swift  
Passport No.    : B12345  
  
Same contact person as previous? (Y)es or (N)o: y  
Enter the passenger No. for same contact person: 1  
  
Contact Person First Name : Shawn  
Contact Person Last Name  : Mendes  
Contact Person Phone No.  : 0125201314  
  
Confirm Check in? (Y)es or (N)o: y  
  
Check-In Successfully ! Thank You !!! ^-^
```

Test Case 7: Print Invoice

```
Invoice Printed!  
Please check your folder :D !  
Press Enter to return to booking menu...
```

justinB_invoice.txt

Welcome to JSJK Airline Company

Departure Flight : KL - Johor
Departure Date : 01/01/2025
Departure Slot : 8:00 A.M.
Return Flight : Penang - KL
Return Date : 05/01/2025
Return Slot : 13:00 P.M.

Passenger 1

Name : Justin Bieber
Passport No. : A12345
Contact Person : Shawn Mendes
Contact Person Mobile No : 0125201314

Passenger 2

Name : Taylor Alison Swift
Passport No. : B12345
Contact Person : Shawn Mendes
Contact Person Mobile No : 0125201314

E. Source Code

```
#include <iostream>

#include <iomanip>

#include <string>

#include <fstream>

#include <cctype>

#include <thread>    // For std::this_thread::sleep_for

#include <chrono>    // For std::chrono::seconds


using namespace std;


struct name {

    string name1;

    string name2;

};


struct info {

    name fullname;

    string phone;

    string email;

    string username;

    string password;

};


struct booking {
```

```
    name passengerName;

    int depart_flight;

    string date_depart;

    int time_depart;

    int return_flight;

    string date_return;

    int time_return;
};
```

```
struct checkInDetails {

    name passengerName;

    string passportNum;

    name contactName;

    string contactMobileNum;
};
```

```
struct PaymentCheckInStatus {

    double totalAmount;

    string paymentStatus;

    checkInDetails checkInPassengers[10];

    string checkInStatus;
};
```

```
#define SIZE 100

string currentUser;
```

```

booking flight = { {"Lily Lee Li", "Ling"}, 7, "12/08/2025", 1, 8,
"14/08/2025", 3 };

int noTickets = 3;

void registration();

void readUser(info users[], int& userCount);

int login(info users[], int userCount);


void menu();

void FlightSchedule();

void performBooking();

int readBooking(string, booking[]);

void editBooking(booking[], string);


void payment();

void readPaymentCheckIn(PaymentCheckInStatus&);

void checkIn(PaymentCheckInStatus&);

void printInvoice(booking, PaymentCheckInStatus);


// Validation for Module 3

void display_bankName();

bool isAllLetter(const string&);

bool isValidCardDate(const string&);

bool isValidSecurityCode(const string&);

bool isValidPassportNum(const string&);

int getValidchoice(int&);

```

```

bool isValidCardNum(const string&);

int getValidBankchoice(int&);


// Decoration

void printhead();

void printThankyou();


int main() {

    cout << endl;

    PaymentCheckInStatus UserCheckIn; // Struct to hold payment and
    check-in status

    int choice1, choice2;

    char cont = 'Y';

    do {

        while (true) {

            //Deco and login menu

            printhead();

            cin >> choice1;

            cin.ignore();

            //Check if the choice is a number

            if (cin.fail()) {

```

```

        cin.clear();

        cin.ignore(1000, '\n');

        cout << "\nInvalid Input. Please enter a number."
<< endl << endl;

        this_thread::sleep_for(chrono::seconds(2));

        system("cls");

    }

    // Check if choice is within range
    else if (choice1 >= 1 && choice1 <= 3)

        break;

    else {

        cout << "\nInvalid Choice. Please enter 1, 2 or
3." << endl << endl;

        this_thread::sleep_for(chrono::seconds(2));

        system("cls");

    }

}

if (choice1 == 1) {

    // Register new user

    registration();

}

```

```

else if (choice1 == 2) {

    // Login process

    int userCount;

    const int maxUsers = 100;

    info users[maxUsers];

    readUser(users, userCount); // Read user info text file

    int error = login(users, userCount); // Check username &
password

    if (error == 0) { // Login successful

        cout << "\nLogin Successful !!!";

        this_thread::sleep_for(chrono::seconds(1));

        system("cls");

        do {

            cout << "==== Booking System =====><

endl;

            cout << "Welcome back, " << currentUser << "

(^_^) " << endl << endl;

            menu(); // Display main booking menu

```

```

        booking bookings[SIZE];

        noTickets = readBooking(currentUser,
bookings); // Load user booking

        readPaymentCheckIn(UserCheckIn);

        // Validate booking menu input
        while (true) {

            cout << "\nYour Choice: ";

            cin >> choice2;

            // Check for non-integer input
            if (cin.fail()) {

                cin.clear();

                cin.ignore(1000, '\n');

                cout << "Invalid choice. Please
enter a number between 1 and 6 !!!\n";

            }

            this_thread::sleep_for(chrono::seconds(2));

            system("cls");

            cout << "===== Booking System
===== " << endl;

            cout << "Welcome back, " <<
currentUser << " (^_^) " << endl << endl;

            menu(); // re-display menu

            continue;

```

```

    }

    // Check if input is within valid
range
    if (choice2 < 1 || choice2 > 6) {
        cout << "Invalid choice. Please
choose between 1 and 6 !!!\n";

        this_thread::sleep_for(chrono::seconds(2));

        system("cls");

        cout << "=====  

===== Booking System  

===== " << endl;

        cout << "Welcome back, " <<
currentUser << " (^_^) " << endl << endl;

        menu(); // re-display menu

        continue;
    }

    break; // Valid input
}

if (choice2 != 6) {

    this_thread::sleep_for(chrono::seconds(1));

    system("cls");

}

if (choice2 == 1) {

```

```

        performBooking();
    }
    else if (choice2 == 2) {
        booking bookings[SIZE];
        editBooking(bookings, currentUser);
    }
    else if (choice2 == 3) {
        if (noTickets == 0) {
            cout << "No booking available"
<< endl;

            cout << "Please book your flight
ticket(s) ! Thank You" << endl;
        }
        else {
            //assign one of the passengers
into global struct flight

            flight = bookings[0];

            payment();
        }
    }
    else if (choice2 == 4) {
        //read the file again, and assign
value, in case there is any update in the file

        readPaymentCheckIn(UserCheckIn);
        checkIn(UserCheckIn); // Perform
check-in

```

```

    }

    else if (choice2 == 5) {

        if (noTickets == 0) {

            cout << "No Booking Found,
Kindly proceed to make booking" << endl;

            cout << "Thank You !!" << endl;

        }

        else {

            readPaymentCheckIn(UserCheckIn);

            printInvoice(flight,
UserCheckIn);

            cout << "Invoice Printed! " <<
endl << endl;

            cout << "Please check your
folder :D !" << endl;

        }

    }

    else if (choice2 == 6) {

        break; // Exit booking system and
return to main menu

    }

    // Prompt to return to menu

    cout << "\nPress Enter to return to booking
menu...";

    cin.ignore();

```

```

cin.get();

//create empty file with NULL value, if user
make or edit booking

if (choice2 == 1 || choice2 == 2) {

    ofstream outFile(currentUser +
"_paymentCheckIn.txt");

    ofstream outFile1(currentUser +
"_invoice.txt");

    //write NULL into _paymentCheckIn.txt
    outFile << 0 << endl;
    outFile << "Not Paid" << endl;

    for (int i = 0; i < noTickets * 6;
i++)

        outFile << "NULL" << endl;

    outFile << "Not Check";

    //write NULL into _invoice.txt
    outFile1 << "Welcome to JSJK Airline
Company" << endl;

    outFile1 << "NULL";

    outFile.close();

```

```

        outFile1.close();

    }

    system("cls");
} while (true);
}

else if (error == 1) {

    // Username not found

    cout << "\nUsername NOT FOUND!" << endl;

    cout << "If you have not registered yet, please
register first o.o !!!" << endl;

}

else if (error == 2)

    // Wrong password

    cout << "Username correct, password INCORRECT!" <<
endl;

}

else if (choice1 == 3) {

    // Exit program

    printThankyou();

```

```

        return 0;
    }

    // Ask user if they want to continue
    if (choice1 == 1 || choice1 == 2) {

        cout << "\nDo you want to continue to register/login?
(Y)es or (N)o: ";

        cin >> cont;

        cont = toupper(cont);

        if (cont == 'N') {
            printThankyou();
            return 0;
        }
    }

    system("cls");

} while (cont == 'Y');
}

```

```

void FlightSchedule() {

    cout << "==== Flight Available =====\n" << endl;

    cout << "1. KL - Penang    --> RM200" << endl;

```

```

cout << "2. Penang - KL    --> RM200" << endl;
cout << "3. KL - Johor    --> RM200" << endl;
cout << "4. Johor - KL     --> RM200" << endl;
cout << "5. KL - Singapore --> RM250" << endl;
cout << "6. Singapore - KL --> RM250" << endl;
cout << "7. KL - Bangkok   --> RM300" << endl;
cout << "8. Bangkok - KL    --> RM300" << endl;

cout << "\nAvailable Departure/Return Time Slots:" << endl;
cout << "\n1. 08:00 A.M." << endl;
cout << "2. 13:00 P.M." << endl;
cout << "3. 18:00 P.M." << endl;
cout << "4. 23:00 P.M." << endl;
}

void menu() {

    cout << "1. Book Flight Ticket(s)" << endl;
    cout << "2. Edit Booking" << endl;
    cout << "3. Perform Transaction" << endl;
    cout << "4. Check In Flight" << endl;
    cout << "5. Print Invoice" << endl;
    cout << "6. Quit" << endl;

}

```

```

void registration() {

    info user;

    system("cls");

    // Display registration heading
    cout << "===== Register =====" << endl;
    cout << "Please enter these details to register!!!" << endl << endl;

    cout << "First Name    : ";
    getline(cin, user.fullname.name1); //struct in struct
    cout << "Last Name     : ";
    getline(cin, user.fullname.name2);
    cout << "Mobile Number : ";
    cin >> user.phone;
    cout << "Email          : ";
    cin >> user.email;
    cin.ignore();
    cout << "Username       : ";
    getline(cin, user.username);

    // Display password requirements
    cout << "\nPassword (Must include these requirements) " << endl;
    cout << "A) 8 Characters" << endl;
    cout << "B) 1 Symbol" << endl;
    cout << "C) 1 Uppercase" << endl;

```

```

cout << "D) 1 Lowercase" << endl;

cout << endl;


// Start password validation loop
do {
    int upper = 0, digit = 0, symbol = 0, error = 0;

    cout << "Password: ";
    getline(cin, user.password);

    // Check minimum length
    if (user.password.length() < 8) {
        cout << "Password too short. Must be at least 8
characters." << endl << endl;
        continue;
    }

    // Count number of uppercase, digits and symbols
    for (int i = 0; i < user.password.length(); i++) {
        if (isupper(user.password[i]))
            upper++;

        if (isdigit(user.password[i]))
            digit++;

        if (ispunct(user.password[i]))

```

```

        symbol++;
    }

    // Check if any required element is missing
    if (upper == 0 || digit == 0 || symbol == 0) {

        cout << "Please include ";

        if (upper == 0) {
            cout << "one Uppercase";
            error++;
        }

        if (digit == 0) {
            if (error > 0)
                cout << " & ";

            cout << "one Number";
            error++;
        }

        if (symbol == 0) {
            if (error > 0)
                cout << " & ";

            cout << "one Symbol";

```

```

        error++;

    }

    cout << "." << endl << endl;

    continue; // Repeat input if password is invalid

}

// Valid password, exit loop

break;

} while (true); // Loop until password valid

cout << "\nRegistration Successfull !! " << endl;

// Save user details to user.txt
ofstream outFile("user.txt", ios::app);

outFile << user.fullname.name1 << endl;
outFile << user.fullname.name2 << endl;
outFile << user.phone << endl;
outFile << user.email << endl;
outFile << user.username << endl;
outFile << user.password << endl;

outFile.close();

}

```

```

void readUser(info users[], int& userCount) {

    // Open file containing user data
    ifstream inFile("user.txt");

    const int maxUsers = 100;
    userCount = 0;

    // Loop and read user info line by line
    for (int i = 0; i < maxUsers; i++) {

        // Read the first line (first name)
        // If it fails, end of file reached
        if (!getline(inFile, users[i].fullname.name1))
            break;

        getline(inFile, users[i].fullname.name2);
        getline(inFile, users[i].phone);
        getline(inFile, users[i].email);
        getline(inFile, users[i].username);
        getline(inFile, users[i].password);

        // Increment the count of users successfully read
        userCount++;

    }

    inFile.close();
}

```

```

int login(info users[], int userCount) {

    string username1, password1;

    system("cls");

    cout << "=====  
Login =====<
" << endl;

    cout << "Username : "<
;

    getline(cin, username1);

    cout << "Password : "<
;

    getline(cin, password1);

    // Loop through the list of registered users
    for (int i = 0; i < userCount; i++) {

        // Check if matching a stored username
        if (users[i].username == username1) {

            // // If the username matches, check the password
            if (users[i].password == password1) {

                // Set the current user to global variable
                currentUser = users[i].username;

                return 0; // Successful login
            }

            else

```

```

        return 2; // Incorrect password
    }

}

return 1; // Username not found
}

void performBooking() {

    booking b[SIZE]; // Array to store booking info

    FlightSchedule();

    cout << "\nNumber of passenger(s): ";
    cin >> noTickets;
    cin.ignore();

    // Check for invalid number of passengers
    if (noTickets <= 0 || noTickets > SIZE) {
        cout << " Invalid number of passengers.\n";
        return; // Exit the function if input is invalid
    }

    // Open file for appending booking data
    ofstream outfile(currentUser + "_Booking.txt");

    // Check if the file is successfully opened

```

```

if (!outfile) {
    cout << " Error opening file for writing.\n";
    return;
}

cout << "\nPlease provide the details as below:\n";

// Collect booking details for each passenger
for (int i = 0; i < noTickets; i++) {

    cout << "\nPassenger " << i + 1 << endl;
    cout << "First Name                : ";
    getline(cin, b[i].passengerName.name1);

    cout << "Last Name                 : ";
    getline(cin, b[i].passengerName.name2);

    cout << endl;

    cout << "Departure Flight (1-8)         : ";
    cin >> b[i].depart_flight;

    while (b[i].depart_flight < 1 || b[i].depart_flight > 8) {
        cout << "Invalid flight, Please enter again !!!" <<
endl;

        cout << "\nDeparture Flight (1-8)         : ";

```

```

        cin >> b[i].depart_flight;
    }

    cout << "Departure Date (DD/MM/YYYY) : ";
    cin >> b[i].date_depart;

    cout << "Departure Time Slot (1-4) : ";
    cin >> b[i].time_depart;

    while (b[i].time_depart < 1 || b[i].time_depart > 8) {
        cout << "Invalid Time Slot, Please enter again !!!" <<
endl;

        cout << "\nDeparture Time Slot (1-4) : ";
        cin >> b[i].time_depart;
    }

    cout << endl;

    cout << "Return Flight (1-8) : ";
    cin >> b[i].return_flight;

    while (b[i].return_flight < 1 || b[i].return_flight > 8) {
        cout << "Invalid flight, Please enter again !!!" <<
endl;

        cout << "Return Flight (1-8) : ";
        cin >> b[i].return_flight;
    }

```

```

cin.ignore();

cout << "Return Date (DD/MM/YYYY)      : ";
cin >> b[i].date_return;

cout << "Return Time Slot (1-4)        : ";
cin >> b[i].time_return;

while (b[i].time_return < 1 || b[i].time_return > 8) {
    cout << "Invalid Time Slot, Please enter again !!!" <<
endl;

    cout << "\nDeparture Time Slot (1-4)    : ";
    cin >> b[i].time_return;
}

cin.ignore();

// Save each booking to file
outfile << b[i].passengerName.name1 << endl
    << b[i].passengerName.name2 << endl
    << b[i].depart_flight << endl
    << b[i].date_depart << endl
    << b[i].time_depart << endl
    << b[i].return_flight << endl
    << b[i].date_return << endl
    << b[i].time_return << endl;

```

```

    }

    // Close file

    outfile.close();

    cout << "\nBooking Completed !!!\n";

}

int readBooking(string currentUser, booking bookings[]) {

    // Open user's booking file for reading

    ifstream infile(currentUser + "_Booking.txt");

    int total = 0; // Counter for no. of bookings read

    while (true) {

        // Read each booking field

        // If any read fails, break the loop

        if (!getline(infile, bookings[total].passengerName.name1))
break;

        if (!getline(infile, bookings[total].passengerName.name2))
break;

        if (!(infile >> bookings[total].depart_flight)) break;

        infile.ignore();

        if (!getline(infile, bookings[total].date_depart)) break;

        if (!(infile >> bookings[total].time_depart)) break;

```

```

        infile.ignore();

        if (!(infile >> bookings[total].return_flight)) break;
        infile.ignore();

        if (!getline(infile, bookings[total].date_return)) break;
        if (!(infile >> bookings[total].time_return)) break;
        infile.ignore();

        total++; // Successfully read one booking

        if (total >= SIZE) break; // Prevent overflow beyond array
size
    }

    infile.close();

    return total; // Return total bookings read
}

void editBooking(booking b[], string currentUser) {

    // Read current bookings from file
    int total = readBooking(currentUser, b);

    cout << "=====  

    // If no data found, show message

```

```

if (total == 0) {
    cout << "\nNo booking found. Please book tickets first.\n";
    return;
}

// If user wants to change passenger names
char changeName;

cout << "\nChange passenger name? (Y)es or (N)o: ";
cin >> changeName;
changeName = tolower(changeName);

if (changeName == 'y') {

    cin.ignore();

    for (int i = 0; i < total; i++) {

        cout << "\nPassenger " << i + 1 << " current name: "
            << b[i].passengerName.name1 << " " <<
b[i].passengerName.name2 << endl;

        cout << "New First Name : ";
        getline(cin, b[i].passengerName.name1);

        cout << "New Last Name : ";
        getline(cin, b[i].passengerName.name2);

    }
}

```

```

        cout << "Passenger name changed successfully !!!" << endl;

        this_thread::sleep_for(chrono::seconds(1));

        system("cls");
    }

    else {

        this_thread::sleep_for(chrono::seconds(1));

        system("cls");
    }

    // Begin loop to edit specific fields

    do {

        cout << "==== Edit Booking ===== " << endl;

        cout << "\nSelect item to amend:\n";

        cout << "1. Departure Flight\n2. Departure Date\n3. Departure
Slot\n";

        cout << "4. Return Flight\n5. Return Date\n6. Return Slot\n7.
Exit\n\n";

        int choice;

        cout << "Select: ";

        cin >> choice;

        cout << endl;

        // Validate user input

```

```

        if (choice < 1 || choice > 7) {
            cout << "Invalid choice. Please enter between 1 to 7 !!!
o.0 \n";

            this_thread::sleep_for(chrono::seconds(3));
            system("cls");
            continue; // Skip loop and go back to menu
        }

        // Show old value for preview if not exiting
        if (choice != 7)
            cout << "Follow the details of 1st Passenger (" <<
b[0].passengerName.name1
            << " " << b[0].passengerName.name2 << ")" << endl;

        // Prompt for updated info based on user choice
        if (choice == 1) {
            cout << "Old Departure Flight: " << b[0].depart_flight
<< endl;

            cout << "New Departure Flight: ";
            cin >> b[0].depart_flight;
        }
        else if (choice == 2) {
            cout << "Old Departure Date: " << b[0].date_depart <<
endl;

            cout << "New Departure Date: ";
            cin >> b[0].date_depart;
        }
    }
}

```

```

else if (choice == 3) {
    cout << "Old Departure Slot: " << b[0].time_depart <<
endl;

    cout << "New Departure Slot: ";
    cin >> b[0].time_depart;
}

else if (choice == 4) {
    cout << "Old Return Flight: " << b[0].return_flight <<
endl;

    cout << "New Return Flight: ";
    cin >> b[0].return_flight;
}

else if (choice == 5) {
    cout << "Old Return Date: " << b[0].date_return << endl;
    cout << "New Return Date: ";
    cin >> b[0].date_return;
}

else if (choice == 6) {
    cout << "Old Return Slot: " << b[0].time_return << endl;
    cout << "New Return Slot: ";
    cin >> b[0].time_return;
}

else if (choice == 7) {
    return;// Exit edit mode
}

```

```

// Apply change to all passengers
for (int i = 0; i < total; i++) {

    b[i].depart_flight = b[0].depart_flight;

    b[i].date_depart = b[0].date_depart;

    b[i].time_depart = b[0].time_depart;

    b[i].return_flight = b[0].return_flight;

    b[i].date_return = b[0].date_return;

    b[i].time_return = b[0].time_return;

}


// Confirmation before saving

char confirmation;

cout << "\nConfirm Editing? (Y)es or (N)o: ";

cin >> confirmation;

confirmation = tolower(confirmation);


if (confirmation == 'y') {

    ofstream outfile(currentUser + "_Booking.txt");

    for (int i = 0; i < total; i++) {

        outfile << b[i].passengerName.name1 << endl

            << b[i].passengerName.name2 << endl

            << b[i].depart_flight << endl

            << b[i].date_depart << endl

            << b[i].time_depart << endl

            << b[i].return_flight << endl

            << b[i].date_return << endl
    }
}

```

```

        << b[i].time_return << endl;

    }

    outfile.close();

    cout << "Booking successfully updated and saved!" <<
endl;

    }

    cout << "\nPress Enter to return to Editing menu..." << endl;

    cin.ignore();

    cin.get();

    cout << "Returning to Editing Menu..." << endl;

    this_thread::sleep_for(chrono::seconds(1));

    system("cls");

    } while (true); // Loop until user choose to exit
}

void payment() {

    char confirm;

    int choice;

    string name, cardDate, secureCode, cardNumber;

    // Ticket prices for each route

    int price[] = { 200,200,200,200,250,250,300,300 };

```

```

    string route_option[] = { "KL - Penang","Penang - KL","KL -
Johor","Johor - KL","KL - Singapore",

    "Singapore - KL","KL - Bangkok","Bangkok - KL" };

    string timeSlot[] = { "8:00 A.M.,"13:00 P.M.,"18:00 P.M.,"23:00
P.M." };


    // Departure and return route names and prices

    string depart_route = route_option[flight.depart_flight - 1];
    string return_route = route_option[flight.return_flight - 1];

    int price_depart = price[flight.depart_flight - 1];
    int price_return = price[flight.return_flight - 1];


    //Display confirmation details

    cout << "==== Booking Details====" << endl;
    cout << "Total Flight Ticket(s) : " << noTickets << endl;


    cout << "\nDeparture Flight" << endl;

    cout << "Date          : " << flight.date_depart << endl;
    cout << "Depart Route : " << depart_route << endl;
    cout << "Time          : " << timeSlot[flight.time_depart - 1] <<
endl;


    cout << "\nReturn Flight" << endl;

    cout << "Date          : " << flight.date_return << endl;
    cout << "Return Route : " << return_route << endl;
    cout << "Time          : " << timeSlot[flight.time_return - 1] <<
endl;

```

```

// Calculate total amount

int amount = (price_depart + price_return) * noTickets;

cout << "\nTotal Payment : " << "(RM" << price_depart << " + RM " <<
price_return << ")" <<

    " x " << noTickets << " = RM" << amount << endl;

// Ask for payment method

cout << "\nPlease Choose Your Payment Method " << endl;

cout << "\n1. Credit/Debit Card" << endl;

cout << "2. Bank Transfer" << endl;


choice = getValidchoice(choice);


system("cls");

cout << "==== Payment =====" << endl;


// If user chooses credit/debit card

if (choice == 1) {

    cout << "Total Amount      : RM" << amount << endl << endl;

    cout << "Card Holder Name : ";

    getline(cin, name);

    cout << "Card Number      : ";

```

```

cin >> cardNumber;

while (!isValidCardNum(cardNumber)) {
    cin >> cardNumber;
}

// Validate card expiry date
cout << "\nCard Valid Date (MM/YYYY): ";
cin >> cardDate;
while (!isValidCardDate(cardDate)) {
    cout << "Invalid Date, Please enter again !!!" << endl;
    cout << "\nCard Valid Date (MM/YYYY): ";
    cin >> cardDate;
}

// Validate security code
cout << "\nSecurity Code : ";
cin >> secureCode;
while (!isValidSecurityCode(secureCode)) {
    cout << "Invalid Security Code, Please enter 3 digit
only !!! " << endl;
    cout << "\nSecurity Code : ";
    cin >> secureCode;
}

// Confirm transaction
cout << "\nConfirm Transaction? (Y)es or (N)o: ";
cin >> confirm;

```

```

        cin.ignore();

    }

    else {

        // Bank Transfer Option

        string UserName, UserPassword, tac;

        int bankOpt;


        display_bankName(); // Function to show bank names

        bankOpt = getValidBankchoice(bankOpt);


        cout << "\nPlease enter your bank details :0" << endl;


        // Get and validate username

        cout << "Username    : ";

        getline(cin, UserName);

        while (!isAllLetter(UserName)) {

            cout << "Invalid name. \nplease enter again ^_^ " <<

endl;

            cout << "Username : ";

            getline(cin, UserName);

        }


        cout << "Password    : ";

        getline(cin, UserPassword);

```

```

        cout << "Payment      : RM" << amount << endl << endl;

        cout << "Confirm and request TAC (Y)es or (N)o: ";

        cin >> confirm;

        if (tolower(confirm) == 'y') {

            cout << "Enter TAC : ";

            cin >> tac;

            while (!isValidSecurityCode(tac)) {

                cout << "Invalid TAC. \nPlease enter again : ";

                cin >> tac;

            }

            cin.ignore();

        }

    }

    // If payment is confirmed, write payment status to file
    if (tolower(confirm) == 'y') {

        ofstream outFile;

        outFile.open(currentUser + "_paymentCheckIn.txt");

        outFile << amount << endl;

        outFile << "Paid" << endl;

        // Fill remaining lines with placeholder data

```

```

        for (int i = 0; i < noTickets * 6; i++) {
            outFile << "NULL\n";
        }

        outFile << "Not Check"; // Check-in status

        outFile.close();

        cout << "Payment Sucessfully ^-^ !!" << endl;
    }
}

void readPaymentCheckIn(PaymentCheckInStatus& UserCheckIn) {

    // Open input file named after the current user with
    "_paymentCheckIn.txt"

    ifstream inFile(currentUser + "_paymentCheckIn.txt");

    // Check if the file was opened successfully
    if (!inFile.fail()) {

        // Read total amount from the file and store it in the
        UserCheckIn

        inFile >> UserCheckIn.totalAmount;

        inFile.ignore();

        // Read the payment status of the user

```

```

        getline(inFile, UserCheckIn.paymentStatus);

        // Loop through each ticket
        for (int i = 0; i < noTickets; i++) {

            getline(inFile,
UserCheckIn.checkInPassengers[i].passengerName.name1);

            getline(inFile,
UserCheckIn.checkInPassengers[i].passengerName.name2);

            inFile >> UserCheckIn.checkInPassengers[i].passportNum;

            inFile.ignore();

            getline(inFile,
UserCheckIn.checkInPassengers[i].contactName.name1);

            getline(inFile,
UserCheckIn.checkInPassengers[i].contactName.name2);

            inFile >>
UserCheckIn.checkInPassengers[i].contactMobileNum;

            inFile.ignore();

        }

        // Read the overall check-in status of the user
        getline(inFile, UserCheckIn.checkInStatus);

        inFile.close();

    }
}

```

```

void checkIn(PaymentCheckInStatus& UserCheckIn) {

    char choice;

    int idx_samecontact;

    cout << "=====  

    Check In ===== " << endl << endl;

    //check the file "Paid" and "Not check" in order to checkin

    if (UserCheckIn.paymentStatus == "Paid" && UserCheckIn.checkInStatus  

    == "Not Check") {

        cout << "Payment Status: Paid" << endl;

        cout << "Proceed to check in =====>" << endl;

        cin.ignore();

        // Loop through all tickets (passengers)

        for (int i = 0; i < noTickets; i++) {

            cout << "\nPassenger " << i + 1 << endl << endl;

            cout << "First Name    : ";

            getline(cin,  

            UserCheckIn.checkInPassengers[i].passengerName.name1);

            cout << "Last Name      : ";

```

```

        getline(cin,
UserCheckIn.checkInPassengers[i].passengerName.name2);

        // Get passport number and validate it
        cout << "Passport No. : ";

        cin >> UserCheckIn.checkInPassengers[i].passportNum;

        while
(!isValidPassportNum(UserCheckIn.checkInPassengers[i].passportNum)) {

            cout << "\nPlease enter again: ";

            cin >>
UserCheckIn.checkInPassengers[i].passportNum;

        }

        cin.ignore();

        if (i != 0) { //check that first passenger is not
allowed to copy the contact info from the previous one

            cout << "\nSame contact person as previous? (Y)es
or (N)o: ";

            cin >> choice;

            cin.ignore();

            // If user chooses to copy contact info from a
previous passenger

            if (tolower(choice) == 'y') {

                cout << "Enter the passenger No. for same
contact person: ";

                cin >> idx_samecontact;

```

```

        cin.ignore();

        // Copy contact info

        UserCheckIn.checkInPassengers[i].contactName.name1 =

        UserCheckIn.checkInPassengers[idx_samecontact -
1].contactName.name1;

        UserCheckIn.checkInPassengers[i].contactName.name2 =

        UserCheckIn.checkInPassengers[idx_samecontact -
1].contactName.name2;

        UserCheckIn.checkInPassengers[i].contactMobileNum =

        UserCheckIn.checkInPassengers[idx_samecontact - 1].contactMobileNum;

        //display contact info

        cout << "\nContact Person First Name : ";

        cout <<
UserCheckIn.checkInPassengers[i].contactName.name1 << endl;

        cout << "Contact Person Last Name      : ";

        cout <<
UserCheckIn.checkInPassengers[i].contactName.name2 << endl;

        cout << "Contact Person Phone No.      : ";

        cout <<
UserCheckIn.checkInPassengers[i].contactMobileNum << endl;

```

```

                                continue; // Skip manual contact info input
for this passenger

                                }

                                }

                                // Get the contact person details if they don't want to
copy

                                cout << "\nContact Person First Name    : ";

                                getline(cin,
UserCheckIn.checkInPassengers[i].contactName.name1);

                                cout << "Contact Person Last Name      : ";

                                getline(cin,
UserCheckIn.checkInPassengers[i].contactName.name2);

                                cout << "Contact Person Phone No.      : ";

                                cin >>
UserCheckIn.checkInPassengers[i].contactMobileNum;

                                cin.ignore();

                                }

                                cout << "\nConfirm Check in? (Y)es or (N)o: ";

                                cin >> choice;

                                //update the file with passengers info and the check in status

                                if (tolower(choice) == 'y') {

```

```

ofstream outFile;

outFile.open(currentUser + "_paymentCheckIn.txt");

// Write updated check-in details to the file
outFile << UserCheckIn.totalAmount << endl;
outFile << UserCheckIn.paymentStatus << endl;

for (int i = 0; i < noTickets; i++) {

    outFile <<
UserCheckIn.checkInPassengers[i].passengerName.name1 << endl;

    outFile <<
UserCheckIn.checkInPassengers[i].passengerName.name2 << endl;

    outFile <<
UserCheckIn.checkInPassengers[i].passportNum << endl;

    outFile <<
UserCheckIn.checkInPassengers[i].contactName.name1 << endl;

    outFile <<
UserCheckIn.checkInPassengers[i].contactName.name2 << endl;

    outFile <<
UserCheckIn.checkInPassengers[i].contactMobileNum << endl;

}

// Mark check-in as completed in the file
outFile << "Checked";

```

```

        outFile.close();

        cout << "\nCheck-In Successfully ! Thank You !!! ^-^" <<
endl;

    }

}

// If the user has already checked in
else if (UserCheckIn.checkInStatus == "Checked") {
    cout << "Check In Completed. Thank you ! ^_^" << endl;
}

// If payment is not made yet
else {
    cout << "Payment Status : Not Paid " << endl;
    cout << "No payment for the flight ticket(s)" << endl;
    cout << "Please proceed to make payment before check in" <<
endl;

    cout << "Thank you :) !!!" << endl;

}
}

```

```

void printInvoice(booking flight, PaymentCheckInStatus UserCheckIn) {

```

```

    // Arrays for route options and time slots

    string route_option[8] = { "KL - Penang","Penang - KL","KL -
Johor","Johor - KL","KL - Singapore",
    "Singapore - KL","KL - Bangkok","Bangkok - KL" };

```

```
    string timeSlot[4] = { "8:00 A.M.", "13:00 P.M.", "18:00 P.M.", "23:00  
P.M." };
```

```
// Open file to write the invoice
```

```
ofstream outFile1;
```

```
outFile1.open(currentUser + "_invoice.txt");
```

```
outFile1 << "Welcome to JSJK Airline Company" << endl;
```

```
outFile1 << "\nDeparture Flight : " <<  
route_option[flight.depart_flight - 1] << endl;
```

```
outFile1 << "Departure Date : " << flight.date_depart << endl;
```

```
outFile1 << "Departure Slot : " << timeSlot[flight.time_depart -  
1] << endl;
```

```
outFile1 << "Return Flight : " <<  
route_option[flight.return_flight - 1] << endl;
```

```
outFile1 << "Return Date : " << flight.date_return << endl;
```

```
outFile1 << "Return Slot : " << timeSlot[flight.time_return -  
1] << endl;
```

```
// Loop through each ticket to write passenger details
```

```
for (int i = 0; i < noTickets; i++) {
```

```
    // Write a blank line for separation between passengers
```

```
    outFile1 << endl;
```

```
    outFile1 << "Passenger " << i + 1 << endl;
```

```

        outFile1 << "Name                      : ";

        outFile1 <<
UserCheckIn.checkInPassengers[i].passengerName.name1 << " ";

        outFile1 <<
UserCheckIn.checkInPassengers[i].passengerName.name2 << endl;


        outFile1 << "Passport No.                : " <<
UserCheckIn.checkInPassengers[i].passportNum << endl;


        outFile1 << "Contact Person                : " <<
UserCheckIn.checkInPassengers[i].contactName.name1 << " ";

        outFile1 << UserCheckIn.checkInPassengers[i].contactName.name2
<< endl;


        outFile1 << "Contact Person Mobile No : " <<
UserCheckIn.checkInPassengers[i].contactMobileNum << endl;

    }


    outFile1.close();

}


//validation for module 3
void display_bankName() {

    cout << "----- Bank Name -----" << endl;

    cout << "1.Maybank          5.CIMB Bank" << endl;

    cout << "2.Public Bank      6.OCBC Bank" << endl;

```

```

    cout << "3.AmBank          7.Alliance Bank" << endl;

    cout << "4.HSBC            8.Standard Chartered" << endl << endl;
}

bool isAllLetter(const string& str) {

    // Loop through each character in the string
    for (int i = 0; i < str.length(); i++) {
        char ch = str[i];

        // If the character not a letter or a space
        if (!isalpha(ch) && !isspace(ch))
            return false;
    }

    // If all characters are valid
    return true;
}

bool isValidCardDate(const string& date) {

    // Check if the date is exactly 7 characters long and contains a '/'
    if (date.length() != 7 || date[2] != '/')
        return false;

    // Extract the month and year from the date
    string MM = date.substr(0, 2);
    string YYYY = date.substr(3, 4);
}

```

```

// Check if both month and year consist only of digits
for (int i = 0; i < 2; i++) {
    char ch = MM[i];
    if (!isdigit(ch))
        return false;
}

for (int j = 0; j < 4; j++) {
    char ch = YYYY[j];
    if (!isdigit(ch))
        return false;
}

// Convert the month to an integer
// Validate if it falls between 1 and 12
int month = stoi(MM);
if (month >= 1 && month <= 12)
    return true;
else
    return false;
}

bool isValidSecurityCode(const string& code) {

    // Check if the code length is exactly 3
    if (code.length() != 3) {
        return false;
    }
}

```

```

    }

    // Check if each character in the code is a digit
    for (int i = 0; i < code.length(); i++) {
        char ch = code[i];
        if (!isdigit(ch))
            return false;
    }

    return true; // Code is valid
}

bool isValidPassportNum(const string& passport) {

    // Check if the passport number length is exactly 6 characters
    if (passport.length() != 6) {
        cout << "\nInvalid Passport Number!" << endl;
        cout << "Please include 1 uppercase & 5 digits only !!!" <<
endl;

        return false;
    }

    // Check if the first character is an uppercase letter
    if (!isupper(passport[0])) {
        cout << "\nInvalid Passport Number!" << endl;
        cout << "First character is uppercase !!!" << endl;
        return false;
    }
}

```

```

// Check if the rest of the characters are digits
for (int i = 1; i < passport.length(); i++) {
    if (!isdigit(passport[i]))
        return false;
}

return true;
}

int getValidchoice(int& choice) {
    cout << "\nEnter Choice : ";
    cin >> choice;

    while (true) {

        if (cin.fail()) {
            cin.clear();
            cin.ignore(1000, '\n');
            cout << "Invalid input. Please enter again : ";
            cin >> choice;
            continue;
        }

        if (choice != 1 && choice != 2) {
            cout << "Invalid input. Please enter again : ";
            cin >> choice;
            continue;
        }
    }
}

```

```

        cin.ignore();

        return choice;

    }

}

bool isValidCardNum(const string& cardnum) {

    if (cardnum.length() != 16) {

        cout << "\nInvalid Card Number !" << endl;

        cout << "Please Enter 16 digits !!!" << endl;

        cout << "Card Number      : ";

        return false;

    }

    // Check if the rest of the characters are digits
    for (int i = 1; i < cardnum.length(); i++) {

        if (!isdigit(cardnum[i])) {

            cout << "\nInvalid Card Number." << endl;

            cout << "Please Enter 16 digits !!!" << endl;

            cout << "Card Number      : ";

            return false;

        }

    }

    return true;

}

int getValidBankchoice(int& bankOpt) {

```

```

cout << "\nSelect Bank : ";

cin >> bankOpt;

while (true) {

    if (cin.fail()) {

        cin.clear();

        cin.ignore(1000, '\n');

        cout << "Invalid input. Please enter again : ";

        cin >> bankOpt;

        continue;

    }

    if (bankOpt < 1 || bankOpt > 8) {

        cout << "Invalid input. Please enter again : ";

        cin >> bankOpt;

        continue;

    }

    cin.ignore();

    return bankOpt;

}

}

// Decoration

void printhead() {

```

```

    cout << "    _      _      _
_
" << endl;

    cout << "  /  |  _  /  |      /  |
/  |
" << endl;

    cout << "  $$ | / \ $$ | _____ $$ | _____
_____
_$$ |_
" <<
endl;

    cout << "  $$ |/$ \$$ | /      \ $$ | /      | /      \ /      \ /
\ /      \      / $$ | /      \
"
<< endl;

    cout << "  $$ /$$$  $$ |/$$$$$$  |$$ |/$$$$$$$/
/$$$$$$  |$$$$$  $$$$  |/$$$$$$  |      $$$$/ /$$$$$  |
" << endl;

    cout << "  $$ $$/$$ $$ |$$  $$ |$$ |$$ |      $$ |  $$ |$$ |  $$ |
$$ |$$  $$ |      $$ | _ $$ |  $$ |
" <<
endl;

    cout << "  $$$$/  $$$$ |$$$$$$$$/ $$ |$$ \_____ $$ \_$$ |$$ |  $$ |
$$ |$$$$$$$$/      $$ | /  |$$ \_$$ |
" <<
endl;

    cout << "  $$$/      $$$ |$$      |$$ |$$      |$$  $$/ $$ |  $$ |
$$ |$$      |      $$  $$/ $$  $$/
" <<
endl;

    cout << "  $$$/      $$$/  $$$$$$$/ $$/  $$$$$$$/  $$$$/  $$$/
$$$/  $$$$$$$/      $$$/  $$$$/
" <<
endl;

    cout << endl;

    cout << "
_____
_____
_
_____
_
_
_____
_____
" <<
endl;

    cout << "      /      | /      \      / | / | / |      /      \ /
|/      \ / |      /      | / \ / | /      | /      \ " << endl;

```

```

    cout << "      $$$$$$ |/$$$$$$ |   $$$$$$ |$$ | /$$/
/$$$$$$ |/$$$$$$/ $$$$$$$ $ |$ |   $$$$$$/ $$ \ $$ |$$$$$$$$/
/$$$$$$ |" << endl;

    cout << "      $$ |$$ \_$$/      $$ |$$ |/$$/      $$ |__$$ |
$$ |  $$ |__$$ |$$ |      $$ |  $$$ \$$ |$$ |__  $$ \_$$/ " <<
endl;

    cout << "  __  $$ |$$      \  __  $$ |$$  $$<      $$  $$ |
$$ |  $$  $$< $$ |      $$ |  $$$$  $$ |$$  |  $$      \ " <<
endl;

    cout << "  /  |  $$ | $$$$$$ |/  |  $$ |$$$$$ \      $$$$$$$$ |
$$ |  $$$$$$ |$$ |      $$ |  $$ $$ $$ |$$$$$/      $$$$$$ |" <<
endl;

    cout << "  $$ \_$$ |/  \_$$ |$$ \_$$ |$$ |$$      \      $$ |
$$ | _$$ |_ $$ |  $$ |$$ |_____ _$$ |_ $$ |$$$$ |$$ |_____ /  \_$$ |"
<< endl;

    cout << "  $$  $$/ $$  $$/ $$  $$/ $$ | $$ |      $$ |  $$ |/
$$  |$$ |  $$ |$$      |/ $$  |$$ | $$$ |$$      |$$  $$/ " <<
endl;

    cout << "  $$$$$$/  $$$$$$/  $$$$$$/  $$/  $$/      $$/  $$/
$$$$$$$/  $$/  $$/ $$$$$$$$$/ $$$$$$/  $$/  $$/ $$$$$$$$$/  $$$$$$/  " <<
endl;

    cout << endl;

    cout << endl;

    cout << endl;

    cout << "
                                _____|_____
                                " << endl;

    cout << "
                                -----
                                " << endl;

    cout << "
                                || | ||
                                " << endl;

```

```

        cout << "                                ||---||
                                " << endl;

        cout << "                                '  _  '
                                " << endl;

        cout << "                                -----'  .  '-----
" << endl;

        cout << "                                _____'  _'
'_____ " << endl;

        cout << "          '-----|---|----/  \==][^',_m_,'^][==/  \----|---|-'
'----- " << endl;

        cout << "                                \_/_      ~      \_/_
" << endl;

        cout << endl;

        cout << "===== MENU
===== " << endl;

        cout << "1. Register New User" << endl;

        cout << "2. Login" << endl;

        cout << "3. Quit" << endl;

        cout << "\nYour Choice: ";

}

void printThankyou() {

    system("cls");

    cout << endl;

    cout << "-----
----- " << endl;

```

```

    cout << "|
Airline!!!
    cout << "|
|" << endl;

    cout << "-----
-----" << endl;

    cout << "
    " << endl;

    cout << " / | / | / |
\ / |
" << endl;

    cout << " $$$$$$/ $$ |
$$ \ /$$/
" << endl;

    cout << " $$ | $$ \ / \ / \ $$ | / |
$$ \/$$/ / \ / | / |" << endl;

    cout << " $$ | $$$$$$ | $$$$$$ | $$$$$$ | $$ |_/$$/
$$ $$/ /$$$$$ |$$ | $$ |" << endl;

    cout << " $$ | $$ | $$ | / $$ |$$ | $$ |$$ $<
$$$$/ $$ | $$ |$$ | $$ |" << endl;

    cout << " $$ | $$ | $$ |/$$$$$$ |$$ | $$ |$$$$$ \
$$ | $$ \_$$ |$$ \_$$ |" << endl;

    cout << " $$ | $$ | $$ |$$ $$ |$$ | $$ |$$ | $$ |
$$ | $$ $$/ $$ $$/ " << endl;

    cout << " $$/ $$/ $$/ $$$$$$/ $$/ $$/ $$/ $$/
$$/ $$$$$/ $$$$$/ " << endl;

    cout << "
" << endl;

    cout << " / \ / | / |
" << endl;

    cout << " /$$$$$ |
" << endl;

```

```

    cout << "  $$ | _$$/ /      \ /      \ /      $$ |$$      \ / | /
| /      \                               " << endl;

    cout << "  $$ |/      |/$$$$$$ |/$$$$$$ |/$$$$$$ |$$$$$$$ |$$ |
$$ |/$$$$$$ |                               " << endl;

    cout << "  $$ |$$$$ |$$ |  $$ |$$ |  $$ |$$ |  $$ |$$ |  $$ |$$ |
$$ |$$  $$ |                               " << endl;

    cout << "
$$ \_$$ |$$ \_$$ |$$ \_$$ |$$ \_$$ |$$ |__$$ |$$ \_$$ |$$$$$$$$$/
" << endl;

    cout << "  $$  $$/ $$  $$/ $$  $$/ $$  $$ |$$  $$/
$$  $$ |$$  |                               " << endl;

    cout << "  $$$$$$/  $$$$$$/  $$$$$$/  $$$$$$/ $$$$$$/
$$$$$$$$ | $$$$$$/                               " << endl;

    cout << "
\_$$ |                               /
\_$$ |                               " << endl;

    cout << "
$$  $$/                               " << endl;

    cout << "
$$$$$/                               " << endl;

}

```